

DEPARTMENT OPERATING SYSTEM

Custom Software & Enterprise Solutions Department Handbook

From zero to professional: business systems, engineering, client delivery, costing, templates and onboarding

VEL Software Solutions

Document VEL-CSE-001 · Version 1.0 (draft for management approval) · 4 November 2026
Tool versions, vendor prices and platform limits change. Always verify against official provider documentation and pricing calculators before building or quoting.

Document Control

Document	Custom Software & Enterprise Solutions Department Handbook (Module M0)
ID / version	VEL-CSE-001 · v1.0 · draft for management approval
Date	4 November 2026
Owner	Department Head, Custom Software & Enterprise Solutions (to be named at approval)
Classification	Confidential — internal use
Review cycle	Quarterly, and after every closed client project (lessons learned)

Change log

Version	Date	Change	Approved by
1.0	4 November 2026	First issue: handbook, onboarding plan, lifecycle, standards, costing model, templates	

ENGINEER MUST VERIFY

This handbook defines methods, controls and templates. It does not certify any system as secure, compliant, or safe. Independent engineer or auditor review is required before production reliance. Prices in Chapter 8 are internal assumptions, not vendor quotes.

Approval

Role	Name	Signature	Date
Prepared by			
Technical Lead			
Department Head			

Contents

How to Use This Handbook 5
1. Department Service Portfolio 7
2. Employee Teaching Standard 9
3. Training Roadmap: Levels 1–6 12
4. Technology Stack Evaluation 18
5. Mandatory Client Project Lifecycle 21
6. Enterprise Project Folder Structure 37
7. Mandatory Enterprise Engineering Standards 40
8. Costing and Commercial Model 44
9. Employee Roles, Responsibility Matrix and Approvals 53
10. Practical Training Projects and Sign-off 56
11. Official Learning Sources 62
Appendix A — Employee Onboarding Checklist 65
Appendix B — 90-Day Training Plan 67
Appendix C — Business Requirements Template (BRD) 69
Appendix D — Functional Requirements Template (FRD / FRS) 71
Appendix E — Statement of Work (SOW) Template 73
Appendix F — Database Design Checklist 75
Appendix G — Code Review and Security Checklist 77
Appendix H — QA Checklist 78
Appendix I — User Acceptance Testing (UAT) Checklist 80
Appendix J — Deployment Checklist (Staging and Production) 82
Appendix K — Client Handover Checklist 84
Appendix L — Project Costing Model Specification 86
Appendix M — Glossary 88

Lifecycle stage index (Chapter 5)

S01 — Lead qualification 22
S02 — Client discovery 22
S03 — Business process study 23
S04 — Stakeholder identification 23
S05 — AS-IS process documentation 24
S06 — TO-BE process design 24
S07 — Functional requirement gathering 25

S08 — Non-functional requirements 25
S09 — Input/output specification 26
S10 — User roles & permissions 26
S11 — Requirements freeze 27
S12 — SOW & commercial approval 27
S13 — Software architecture 28
S14 — Database architecture 28
S15 — UI/UX design 29
S16 — Development planning 29
S17 — Repository creation 30
S18 — Module-wise development 30
S19 — API & integration development 31
S20 — Database migration (schema) 31
S21 — Security implementation 32
S22 — Testing 32
S23 — User acceptance testing (UAT) 33
S24 — Staging deployment 33
S25 — Production deployment 34
S26 — Data migration (go-live load) 34
S27 — Employee (end-user) training 35
S28 — Client handover 35
S29 — Maintenance & upgrades 36

How to Use This Handbook

This handbook is the single operating manual of the **Custom Software & Enterprise Solutions Department**. It does two jobs: **(1)** it trains a beginner into a professional who can join supervised commercial software projects, and **(2)** it fixes the mandatory lifecycle, standards, templates, costing model and approval gates that every client project follows, from lead qualification to production support.

GOLDEN RULES

1. Do not skip stages. A stage starts only when the previous stage's approval gate is signed.
2. “It works on my laptop” is not “production-ready”. Production-ready means secure, tested, monitored, backed up, restorable, documented and handed over.
3. Requirements are frozen in writing before design and development. Anything after the freeze is a **change request**.
4. Never put passwords, keys or tokens in Git, chat, documents or tickets.
5. Never run destructive commands (`DROP` , `DELETE` without `WHERE` , `rm -rf`) without understanding the effect and a verified backup.
6. Never claim a system is “secure”, “compliant” or “guaranteed”. We offer *readiness support*; certification belongs to authorised auditors.
7. Every production change has a second reviewer. Nobody approves their own work.
8. Never design multi-tenancy unless the client has explicit tenant-isolation requirements — and then never ship it without isolation tests.

Reading path for a beginner

Part A · Foundations (Ch 1–3) → Part B · Technology & lifecycle (Ch 4–5) →
Part C · Architecture & standards (Ch 6–7) → Part D · Business & people (Ch 8–9) →
Part E · Projects & sources (Ch 10–11) → Appendices · Templates

How the module series works

The department curriculum is delivered **one module at a time**. This document is **Module 0 — the Department Handbook and Onboarding Plan**: it defines the whole system and gives each topic a summary pack. Each later module takes one level or lifecycle area and expands it into full lessons using the 14-point teaching structure in Chapter 2.

Module	Scope	Status
M0	Department Handbook, onboarding plan, lifecycle, standards, costing, templates (this document)	Delivered — v1.0

Module	Scope	Status
M1	Level 1 — Business Systems Fundamentals (full lesson pack)	Next
M2	Level 2 — Business Analysis (discovery, process mapping, user stories)	Planned
M3	Level 3 — Software Engineering (C#/.NET 8 track and Python/TypeScript track)	Planned
M4	Level 4 — Database Engineering (PostgreSQL, SQL Server, MySQL)	Planned
M5	Level 5 — Enterprise Architecture	Planned
M6	Level 6 — Professional Engineering (Git, CI/CD, Docker, cloud, monitoring)	Planned

CURRENCY OF INFORMATION

Tool versions, vendor prices and platform limits change. This handbook gives **methods and formulas**; every vendor price used in a quote must be re-verified on the vendor's official pricing page or calculator on the day of quoting and recorded with the date. Prices in worked examples are **assumptions, not vendor quotes**.

1. Department Service Portfolio

Sixteen commercially deliverable service lines. Each can be sold alone or combined into a multi-module programme.

#	Service line	What we deliver	Typical client trigger
1	**ERP development**	Finance, purchase, sales, stock, production and HR modules on one data model with approvals.	“Every department keeps its own Excel sheet.”
2	**CRM development**	Leads, accounts, pipeline, quotes, follow-ups, activity timeline, sales reports.	“We lose leads and don't know who called whom.”
3	**HRMS development**	Employee master, attendance, leave, payroll inputs, appraisals, documents, self-service.	“Leave and payroll are managed on paper.”
4	**Inventory management**	Items, warehouses, batches/serials, GRN, issue, transfer, reorder, stock valuation.	“Stock in the system never matches the shelf.”
5	**Billing & invoicing**	Quotes, GST-ready invoices, receipts, credit notes, payment tracking, reminders.	“Invoices are made in Word and payments are missed.”
6	**Accounting integrations**	Two-way sync with Tally, Zoho Books, QuickBooks etc.; ledger mapping; reconciliation.	“We key every bill twice.”
7	**Hospital & clinic systems**	Registration, appointments, OPD/IPD, billing, pharmacy, lab, records with strict access control.	“Patient files are lost; billing leaks revenue.”
8	**School & college systems**	Admissions, fees, attendance, timetable, exams, results, parent portal.	“Fee collection and results take weeks.”
9	**Project management systems**	Projects, tasks, timesheets, resource plans, milestones, client approvals, profitability.	“We cannot see project profit until it ends.”
10	**Manufacturing management**	BOM, work orders, routing, shop-floor entry, quality checks, material planning.	“We don't know true cost or WIP.”
11	**Logistics management**	Orders, trips, drivers, tracking, proof of delivery, freight billing.	“Dispatch runs on phone calls.”
12	**Booking & appointments**	Slots, resources, reminders, payments, cancellations, calendar sync.	“No-shows and double bookings.”
13	**Document management**	Versioned storage, metadata, search, access rights, retention, e-sign hooks.	“Nobody can find the latest contract.”
14	**Workflow management**	Configurable multi-step approvals, SLAs, escalations, audit trail.	“Approvals travel by email and stall.”
15	**Custom enterprise portals**	Customer, vendor, employee and partner portals over existing systems.	“Customers keep phoning for status.”

#	Service line	What we deliver	Typical client trigger
16	**Legacy modernisation**	Assessment, strangler-pattern migration, re-platforming, data migration, parallel run.	"Our VB6/Access/FoxPro app is a single point of failure."

REGULATED DOMAINS

Hospital, school, payroll and financial systems hold sensitive personal data. We build the controls (access control, audit trails, encryption, consent capture) and say "**compliance readiness support**". We never state a system is compliant or certified; the client's own legal/compliance owner signs that off.

Which service line first?

For a new department, sell in this order of delivery risk: **billing/inventory/booking** (small, clear scope) → **CRM/HRMS/project** (medium) → **ERP/manufacturing/hospital** (large, multi-module). Do not accept an ERP-scale programme until the team has delivered two medium projects and the Technical Lead has signed off the reference architecture.

Official references for this chapter

- Martin Fowler — articles & catalogue — `https://martinfowler.com/`
- OWASP — `https://owasp.org/`

2. Employee Teaching Standard

Every topic in every module is taught with the same 14-point structure. A topic is **not complete** until all fourteen points exist and the employee has been approved on it.

#	Point	What the lesson must contain
1	What it is	A plain-language definition in two sentences, with one real-world example.
2	Why businesses need it	The business problem solved and the cost of not having it.
3	How it works	A diagram or step list of the mechanism — not just the vocabulary.
4	Tools required	Exact tools, versions policy and licence (free/permissive first).
5	Installation	Copy-paste setup on the company standard workstation, with a check command.
6	Implementation	A guided, working example built step by step.
7	Expected result	What the finished output looks like (screen, output, file, test report).
8	Verification	How the employee *proves* it works: commands, tests, queries.
9	Common errors	The five most frequent mistakes with symptom → cause → fix.
10	Security	What can go wrong securely and the control that prevents it.
11	Documentation	What must be written down (README, ADR, runbook, diagram) and where it is stored.
12	Practical exercise	A task with no solution shown, scoped to 2–4 hours.
13	Assessment	Quiz + demonstration with a pass mark (default ≥ 80 %).
14	Approval	Named approver signs the topic in the employee's training record.

PASS RULES

A **level** is passed only when every exercise is demonstrated live to the Technical Lead, every quiz is ≥ 80 %, and the level's capstone is reviewed. Re-sit allowed once per quiz after 3 working days of remedial study.

Worked example A — Database transactions (all 14 points)

Point	Lesson content
1. What	A **transaction** is a group of database changes that either *all* succeed or *all* are undone (atomicity). Example: moving ₹1,000 from account A to B.

Point	Lesson content
2. Why	Without it, a crash between “debit A” and “credit B” loses money. ERP/billing data must always balance.
3. How	`BEGIN` → statements → `COMMIT` (make permanent) or `ROLLBACK` (undo). ACID: Atomic, Consistent, Isolated, Durable. Isolation levels control what concurrent users see.
4. Tools	PostgreSQL 16+ (or SQL Server / MySQL InnoDB), `psql` or Azure Data Studio/DBeaver, .NET 8 with EF Core or Npgsql.
5. Install	`docker run --name pg -e POSTGRES_PASSWORD=<local-dev-only> -p 5432:5432 -d postgres:16` → check: `docker exec pg psql -U postgres -c "select version()"`.
6. Implement	See SQL and C# below: create `accounts`, write a transfer, force a failure, observe rollback.
7. Result	After a forced failure, both balances are unchanged; after success, the total is unchanged.
8. Verify	Run `select sum(balance) from accounts` before/after: must be identical. Add an automated integration test.
9. Errors	Forgetting `COMMIT` (locks held); swallowing exceptions so rollback never happens; long transactions holding locks; mixing two connections in one logical unit; retrying non-idempotent work.
10. Security	Use least-privilege DB accounts; parameterised queries only; never build SQL by string concatenation.
11. Docs	Record the unit-of-work boundary per use case in the architecture doc; note isolation level and retry policy.
12. Exercise	Implement “issue stock to production order” that decrements stock, inserts a movement row and writes an audit row atomically.
13. Assess	10-question quiz + live demo of a rollback test.
14. Approve	Technical Lead / Database Engineer signs “Transactions” in the training record.

```sql
-- PostgreSQL
create table accounts(id int primary key, owner text not null, balance numeric(14,2) not null check
(balance >= 0));
insert into accounts values (1,'A',5000),(2,'B',1000);

begin;
  update accounts set balance = balance - 1000 where id = 1;
  update accounts set balance = balance + 1000 where id = 2;
commit;           -- use ROLLBACK; to undo

select sum(balance) from accounts;   -- must stay 6000
```

```csharp
// C# 12 / .NET 8 + EF Core — one business operation = one transaction
public async Task TransferAsync(int from, int to, decimal amount, CancellationToken ct)
{
    await using var tx = await _db.Database.BeginTransactionAsync(ct);
    try
    {
        var a = await _db.Accounts.SingleAsync(x => x.Id == from, ct);
        var b = await _db.Accounts.SingleAsync(x => x.Id == to, ct);
        if (a.Balance < amount) throw new DomainException("Insufficient balance");
        a.Balance -= amount; b.Balance += amount;
        _db.AuditLog.Add(AuditEntry.For("Transfer", from, to, amount));
        await _db.SaveChangesAsync(ct);
        await tx.CommitAsync(ct);
    }
    catch { await tx.RollbackAsync(ct); throw; }   // never swallow
}
```

Worked example B — Role-based access control (summary)

Point	Lesson content
What / Why	RBAC grants *permissions* to *roles* and roles to *users*. Businesses need it so clerks cannot approve payments and auditors can read but not edit.
How	Authenticate (who) → load roles/claims → authorise each action (`invoice.approve`) on the **server**. UI hiding is cosmetic only.
Implement	ASP.NET Core policies: `[Authorize(Policy = "invoice.approve")]` ; permission table `role_permission` ; deny by default.
Verify	A test per role × sensitive endpoint: allowed roles get 200, others 403; unauthenticated gets 401.
Errors	Checking permission only in the frontend; role names hard-coded across code; forgetting object-level checks (user A reading user B's record — OWASP “Broken Access Control”).
Security	Deny by default; log denials; review privileged roles quarterly; separation of duties (maker ≠ checker).

CONVENTION

In Modules M1–M6 each topic appears as a full 14-point lesson file. In this handbook (M0) the roadmap in Chapter 3 gives the *summary pack* (definition → why → exercise → check) for every topic so that the whole system can be planned and approved first.

Official references for this chapter

- PostgreSQL documentation — `https://www.postgresql.org/docs/`
- ASP.NET Core documentation — `https://learn.microsoft.com/aspnet/core/`
- OWASP Top 10 — `https://owasp.org/www-project-top-ten/`

3. Training Roadmap: Levels 1–6

A beginner reaches supervised-project readiness in six levels (~ 90 days with the plan in Appendix B). Levels 1–2 are shared by everyone; from Level 3 the employee follows a role track (see Chapter 9).

L1 Business systems → L2 Business analysis → L3 Software engineering → L4 Database → L5 Enterprise architecture → L6 Professional engineering

Level 1 — Business Systems Fundamentals

Duration: 2 weeks · **Tools:** Excel/Sheets, draw.io, a notebook

Topic	What it is	Why businesses need it	Exercise & check
How businesses operate	Organisations buy, make/serve, sell, collect cash and report.	Software models these flows; if you don't understand them you build the wrong thing.	Map how a local shop earns a rupee, from supplier to bank.
Departments & workflows	Sales, purchase, stores, production, finance, HR, support — each with hand-offs.	Most defects are hand-off gaps between departments.	Draw the order-to-cash flow of a distributor.
Business processes	A repeatable sequence of steps with inputs, outputs and an owner.	Only a defined process can be automated or measured.	Write one process as numbered steps with owner per step.
Users & roles	Who uses the system and what each is allowed to do.	Drives permissions, screens and training.	List 6 roles for a clinic and their allowed actions.
Records & transactions	A **master** record (customer) vs a **transaction** (invoice).	Correct structure keeps reports and audits valid.	Classify 15 fields as master / transaction / reference.
Approval workflows	A request moves through approvers by rules (amount, role).	Controls spend and fraud; the most requested ERP feature.	Design a 3-level purchase approval with limits.
Reporting	Turning records into decisions: lists, summaries, dashboards.	Clients buy software for the *reports* they get.	Sketch 5 reports a store owner needs and their data fields.
Business rules	Conditions the business enforces (credit limit, GST slab).	Rules live in configuration/code and change often.	Write 10 rules in "IF … THEN …" form.
Functional vs non-functional requirements	What the system does vs how well (speed, security, uptime).	Non-functional gaps cause most production failures.	Classify 20 sample requirements and add 5 missing NFRs.

Common mistakes: treating a spreadsheet column as a requirement; ignoring exceptions (returns, cancellations); mixing master and transaction data.

Capstone: Document one real small business (shop/clinic/tuition centre): departments, 3 workflows, roles, 5 reports.

Assessment: 30-question quiz ≥ 80 % + whiteboard the order-to-cash flow · **Approval:** Business Analyst lead

Level 2 — Business Analysis

Duration: 3 weeks · **Tools:** draw.io/BPMN, Excel, Markdown, Jira/GitHub Issues

Topic	What it is	Why businesses need it	Exercise & check
Client discovery	Structured meetings to learn goals, pain and constraints.	Wrong discovery = wrong product and disputed scope.	Run a mock 45-min discovery; submit notes in the template.
Stakeholder interviews	Questioning owners, users and IT using open, probing questions.	Different stakeholders hold different truths.	Interview a peer; list 3 conflicting statements.
Requirement gathering	Capturing, numbering and validating needs.	Traceable IDs link need → design → test.	Produce 25 numbered requirements (FR-001…).
Process mapping	Visual flow with swimlanes, decisions and exceptions.	Shows waste and automation points.	Map a leave-approval process in BPMN.
AS-IS and TO-BE	Current process vs improved process after the system.	Proves value; fixes scope boundary.	Produce both diagrams and a gap list.
User stories	“As a <role> I want <goal> so that <benefit>.”	Small testable slices of value.	Write 12 stories, each ≤ 3 days of work (INVEST).
Acceptance criteria	Given/When/Then conditions that define “done”.	Basis of UAT and payment milestones.	3 criteria per story, all testable.
Use cases	Actor–system interaction with main and alternate flows.	Covers exceptions developers miss.	Write 3 use cases with 2 alternate flows each.
Scope management	In-scope / out-of-scope / assumptions / dependencies.	Prevents unpaid scope creep.	Write the scope section of a SOW.
Change requests	Formal evaluation (impact on time/cost/risk) before changing scope.	Protects margin and relationship.	Price one change using the rate card (Ch 8).

Common mistakes: solution-first thinking (“we'll use a dashboard”) before problem definition; unmeasurable words (“fast”, “user-friendly”); no owner for each requirement.

Capstone: Full BRD + FRD (Appendix C/D templates) for a 4-module mini ERP with AS-IS/TO-BE maps.

Assessment: Requirement-review defence in front of a panel (scored on a rubric) · **Approval:** Business Analyst lead + Project Manager

Level 3 — Software Engineering

Duration: 4 weeks (track-based) · **Tools:** VS Code / Visual Studio, .NET 8 SDK **or** Python 3.12 + Node 20 LTS, Git, Postman/curl

Topic	What it is	Why businesses need it	Exercise & check
Programming fundamentals	Variables, control flow, functions, collections, errors.	Every later skill depends on it.	Solve 20 graded problems (FizzBuzz → invoice totals).
C# & .NET 8 (or Python / TypeScript)	Typed language and runtime used for most projects.	Strong typing and tooling suit long-lived business systems.	Build a console invoice calculator with unit tests.
Object-oriented programming	Classes, encapsulation, interfaces, inheritance vs composition.	Models business entities and swaps implementations.	Model Customer/Invoice/Payment with invariants.
APIs	HTTP contracts (REST, JSON) between client and server.	Web, mobile and integrations all use APIs.	Design and call 5 endpoints; publish OpenAPI.
Backend architecture	Layers: API → application → domain → infrastructure.	Keeps rules testable and replaceable.	Refactor a controller-heavy app into layers.
Frontend architecture	Components, state, routing, forms, API client.	Users judge the product by the UI.	Build a list/detail/edit screen in React + TypeScript.
Authentication	Proving identity (passwords, OIDC, MFA, JWT/cookies).	Foundation of every control.	Implement login with hashed passwords; no plain text.
Authorisation	Deciding what an identity may do (roles, policies, ownership).	Prevents data exposure (OWASP A01).	Add policy tests for 3 roles.
Error handling	Consistent, safe error responses and exceptions.	Stable systems and safe messages.	Return RFC 7807 problem details; no stack traces to clients.
Logging	Structured events with correlation IDs, no secrets.	Support and audit depend on logs.	Trace one request across logs using a correlation ID.

Common mistakes: business logic in controllers/UI; catching exceptions and ignoring them; logging passwords/tokens; trusting client-side validation.

Capstone: Employee-management API + web UI (Project P1 in Chapter 10).

Assessment: Code review pass (≥ 80 % rubric) + 60 % automated test coverage on domain code · **Approval:** Technical Lead

Level 4 — Database Engineering

Duration: 3 weeks · **Tools:** PostgreSQL 16+, SQL Server Developer/Express, MySQL 8, DBeaver/Azure Data Studio, Docker

Topic	What it is	Why businesses need it	Exercise & check
Relational databases	Tables, rows, keys and relationships.	The core of almost every business system.	Design 6 tables for a school with PK/FK.
SQL	SELECT/INSERT/UPDATE/DELETE, joins, grouping, window functions.	Reports and troubleshooting need SQL fluency.	Write 20 report queries from a sample schema.
PostgreSQL	Open-source RDBMS with rich types and strong standards support.	Default choice for cost and portability.	Install, create roles, restore a dump.
SQL Server	Microsoft RDBMS, common in Windows/.NET shops.	Many clients already own licences.	Port a schema; compare `identity` vs sequences.
MySQL	Popular open-source RDBMS (InnoDB).	Common on shared hosting and PHP legacy.	Note differences in quoting, `AUTO_INCREMENT`, defaults.
Normalisation	Removing duplication (1NF–3NF) so each fact is stored once.	Prevents update anomalies and wrong totals.	Normalise a denormalised invoice sheet to 3NF.
Transactions	ACID units of work.	Money and stock must never be half-updated.	Rollback test (Chapter 2, Example A).
Indexes	Structures that speed lookups at write/storage cost.	Slow screens are usually missing indexes.	Read an `EXPLAIN` plan; add and measure an index.
Migrations	Versioned, repeatable schema changes.	Safe releases across dev/stage/prod.	Add a column with EF Core/Flyway/Alembic; roll forward.
Data integrity	PK, FK, UNIQUE, CHECK, NOT NULL, triggers sparingly.	The database is the last line of defence.	Break each constraint on purpose and show the error.
Backup & recovery	Full/differential/log backups; tested restores; RPO/RTO.	A backup never restored is only a hope.	Backup, drop a table, restore to a point in time.

Common mistakes: no foreign keys “for speed”; storing money as float; `SELECT *` in code; running migrations manually on production; never testing restores.

Capstone: Normalised schema + migrations + seed data + restore drill for the Inventory system (P2).

Assessment: SQL practical (timed) + restore drill witnessed by the Database Engineer · **Approval:** Database Engineer

Level 5 — Enterprise Architecture

Duration: 3 weeks · **Tools:** draw.io / C4 model, Docker Compose, RabbitMQ, Redis, OpenAPI

Topic	What it is	Why businesses need it	Exercise & check
Modular monoliths	One deployable with strict internal module boundaries.	Best default: simple to run, easy to split later.	Split an app into 4 modules with enforced references.
Microservices & trade-offs	Independently deployed services with own data.	Scale/teams benefits vs cost, latency, consistency pain.	Write an ADR justifying monolith vs microservices for 3 cases.
Multi-tenancy	One system serving many customers with isolated data.	SaaS economics — but a major leak risk.	Only with explicit requirement: implement tenant filter + isolation tests.
Role-based access	Roles → permissions → users, plus data scope (branch/department).	Segregation of duties.	Permission matrix for 6 roles × 12 actions.
Audit trails	Immutable who/what/when/before/after records.	Disputes, fraud checks, compliance support.	Add audit on invoice edits; prove it cannot be altered by the app.
Workflow engines	Configurable state machines for approvals.	Business changes flows without redeploying.	Model Draft→Submitted→Approved→Posted with guards.
Notifications	Email/SMS/in-app/webhook events.	Keeps users in the loop; reduces follow-ups.	Outbox pattern: send reliably after commit.
Reporting	Operational reports vs analytical (read replica, warehouse).	Heavy reports must not slow transactions.	Build 3 reports; show query plan and index.
Integration patterns	REST, webhooks, queues, file exchange, idempotency, retries.	Systems never live alone.	Integrate a mock accounting API with retry + idempotency key.
Data migration	Extract → clean → map → load → reconcile.	Go-live fails more from bad data than bad code.	Migrate a messy CSV with a reconciliation report.

Common mistakes: microservices on day one; tenant ID forgotten on one query; audit written by the same code that can be bypassed; migrating without reconciliation totals.

Capstone: Architecture pack for the multi-module ERP prototype (P6).

Assessment: Architecture review board presentation (C4 L1–L3 + 3 ADRs) · **Approval:** Technical Lead + Enterprise Architect (external reviewer if none)

Level 6 — Professional Engineering

Duration: 3 weeks · **Tools:** Git, GitHub, GitHub Actions, Docker, a cloud sandbox with budget alert, OWASP ZAP/Dependency scanners

Topic	What it is	Why businesses need it	Exercise & check
Git & GitHub	Version control, branches, pull requests.	Traceability and safe collaboration.	Feature branch → PR → review → squash merge.
Code reviews	Peer inspection against a checklist.	Cheapest defect-removal step.	Review a seeded-bug PR and find ≥ 8 of 10 issues.
Automated testing	Unit, integration, system, acceptance tests.	Lets us change code without fear.	Test pyramid for one module; coverage report.
Docker	Packaging an app with its runtime.	Same behaviour on every machine.	Multi-stage image, non-root user, health check.
CI/CD	Automatic build-test-deploy on every change.	Fast, repeatable, auditable releases.	GitHub Actions: build → test → scan → image → deploy to staging.
Cloud deployment	Running on AWS/Azure/GCP (PaaS first).	Clients need hosted, backed-up systems.	Deploy to a sandbox with TLS and a custom domain.
Monitoring	Metrics, logs, traces, alerts, uptime checks.	Find outages before the client does.	Dashboard + alert for error rate and latency.
Security	OWASP Top 10, secrets, dependency and image scanning.	A breach ends a startup's reputation.	Fix 5 seeded OWASP issues; secret-scan passes.
Documentation	README, ADRs, runbooks, API docs, user manuals.	Handover and support depend on it.	A new joiner runs the project from README in < 30 min.

Common mistakes: committing secrets; pushing directly to `main`; no rollback plan; alerts nobody owns; tests that pass only on the author's machine.

Capstone: Take the Billing application (P4) from empty repo to monitored staging deployment.

Assessment: Full pipeline demonstration + security checklist · **Approval:** Technical Lead + Project Manager

Official references for this chapter

- .NET documentation — `https://learn.microsoft.com/dotnet/`
- ASP.NET Core documentation — `https://learn.microsoft.com/aspnet/core/`
- SQL Server documentation — `https://learn.microsoft.com/sql/`
- PostgreSQL documentation — `https://www.postgresql.org/docs/`
- React — Learn — `https://react.dev/learn`
- TypeScript documentation — `https://www.typescriptlang.org/docs/`
- Docker Docs — `https://docs.docker.com/`
- GitHub Docs — `https://docs.github.com/`
- OWASP — `https://owasp.org/`

4. Technology Stack Evaluation

Technology is chosen from **five factors**: client environment, target platform, budget, employee skills and long-term maintainability. We never choose a tool because it is fashionable.

Evaluation of each option

Technology	Best fit	Avoid / caution	Skill & cost note
C# / .NET 8 + ASP.NET Core	Long-lived enterprise back-ends, Windows/Microsoft shops, strong typing, excellent tooling and performance.	Teams with no .NET skills and tiny budgets.	Free, open source; Visual Studio Community/VS Code. Default for backend.
WPF	Windows-only desktop apps: billing counters, shop-floor stations, offline-capable tools.	Anything needing browser/mobile access; Linux/macOS users.	Runs only on Windows; use MVVM; keep logic in UI-free libraries.
React + TypeScript	Rich web UIs, large ecosystem, hiring pool; portals and dashboards.	Over-engineering simple CRUD; teams without TS discipline.	Free. Default for web frontend.
Angular	Large teams wanting an opinionated, batteries-included framework.	Small teams needing speed; steep learning curve.	Free; choose when the client already runs Angular.
TypeScript	Typed JavaScript for front-end and Node tooling.	Mixing `any` everywhere defeats its value.	Mandatory for all JS code.
Python	Data work, automation, integrations, AI features, quick prototypes.	Very large strictly-typed domains without type-checking discipline.	Free; use type hints + mypy/ruff.
FastAPI	Fast Python APIs with automatic OpenAPI and validation (Pydantic).	Heavy admin-style apps better served by Django.	Free; good for integration microservices.
PostgreSQL	Default OLTP database: standards-compliant, JSON, extensions, no licence fee.	Clients locked to SQL Server tooling.	Free; managed services are inexpensive.
SQL Server	Client already licensed; deep Microsoft integration; reporting tools.	Licence cost for new, cost-sensitive clients.	Licensing varies by edition — verify on Microsoft's site.
MySQL	Existing LAMP/hosting estates; read-heavy simple workloads.	Complex reporting/constraints vs PostgreSQL.	Free community edition; know the storage-engine (InnoDB) behaviour.

Technology	Best fit	Avoid / caution	Skill & cost note
Redis	Caching, sessions, rate limiting, short-lived locks.	Using it as the primary database for business records.	Cache is disposable by design; never the system of record.
RabbitMQ	Reliable background jobs, integration events, decoupling.	Simple apps where a DB-backed job table suffices.	Adds an operational component — justify in an ADR.
REST APIs	Default API style for web/mobile/integration; OpenAPI documented.	Chatty fine-grained calls; use batching or GraphQL only if justified.	Version URLs/headers; document in OpenAPI.
Docker	Reproducible packaging for dev, CI and deployment.	Windows-GUI apps like WPF (not containerised).	Free for small business per vendor terms — verify current licence terms.
GitHub Actions	CI/CD where code is on GitHub.	Long, expensive builds without caching.	Free quota varies by plan — verify on GitHub's pricing page.
AWS / Azure / GCP	Managed hosting, databases, storage, identity, monitoring.	Over-engineering (Kubernetes) for small apps.	Choose by client's existing relationship, region/latency and your team's skills.

Weighted decision matrix (use in the architecture ADR)

Score each candidate 1–5 per factor, multiply by weight, sum. The **weights below are the department default** and may be changed *only with written justification* in the ADR.

Option	Client environment fit (25%)	Target platform (20%)	Budget (licence + hosting) (15%)	Team skills available (20%)	Long-term maintainability (20%)	Weighted /5
A: .NET 8 + React + PostgreSQL	4	5	5	4	5	**4.55**
B: .NET 8 + WPF + SQL Server	5	2	3	4	4	**3.70**
C: FastAPI + React + PostgreSQL	3	5	5	3	4	**3.90**

Worked check for A: 4×0.25 + 5×0.20 + 5×0.15 + 4×0.20 + 5×0.20 = 1.00 + 1.00 + 0.75 + 0.80 + 1.00 = **4.55**. The scores are illustrative for a typical web ERP; a Windows-only counter application would score B higher on platform fit.

Department default stacks

Scenario	Default stack	Hosting default
Web ERP / CRM / HRMS / portal	ASP.NET Core 8 modular monolith · React + TypeScript · PostgreSQL · Redis (cache only)	Managed PaaS container service + managed PostgreSQL
Microsoft-centric client	ASP.NET Core 8 · Angular or React · SQL Server	Azure App Service + Azure SQL, or client's data centre
Windows counter/billing desktop	WPF (MVVM) · UI-free domain libraries · SQL Server/PostgreSQL · optional sync API	Local/branch server + cloud backup
Integration & data automation	Python 3.12 + FastAPI · RabbitMQ where needed · PostgreSQL	Container on the same platform
Legacy modernisation	Strangler pattern: new .NET services/UI beside the old app, shared database first, then split	Parallel run on staging

RULE: JUSTIFY, DON'T PRESUME

Microservices, Kubernetes, message brokers and multi-tenancy are **not defaults**. Each needs an Architecture Decision Record stating the requirement that forces it, the alternatives, and the cost of operating it.

Official references for this chapter

- .NET documentation — `https://learn.microsoft.com/dotnet/`
- WPF documentation — `https://learn.microsoft.com/dotnet/desktop/wpf/`
- React — Learn — `https://react.dev/learn`
- Angular documentation — `https://angular.dev/`
- FastAPI documentation — `https://fastapi.tiangolo.com/`
- PostgreSQL documentation — `https://www.postgresql.org/docs/`
- MySQL reference manual — `https://dev.mysql.com/doc/`
- Redis documentation — `https://redis.io/docs/`
- RabbitMQ documentation — `https://www.rabbitmq.com/docs`
- Docker Docs — `https://docs.docker.com/`
- GitHub Actions — `https://docs.github.com/actions`
- AWS documentation — `https://docs.aws.amazon.com/`
- Azure documentation — `https://learn.microsoft.com/azure/`
- Google Cloud documentation — `https://cloud.google.com/docs`

5. Mandatory Client Project Lifecycle

Twenty-nine stages in five phases. For every stage this chapter states **tasks, owners, inputs, outputs, templates, review requirements and the approval gate**. No stage starts until the previous gate is signed.

A Discover & Define (1–12) → B Design & Plan (13–17) → C Build (18–21) →
D Verify & Release (22–26) → E Adopt & Sustain (27–29)

Approval gates at a glance

Gate	After stage	Approver(s)	Evidence required
G1 Qualification	1	Department Head	Scorecard ≥ 18/25
G2 Requirements freeze	11	Client sponsor, Department Head	Signed BRD/FRS/NFR/I-O/permission matrix
G3 Commercial approval	12	Department Head, client	Signed SOW, margin ≥ target, advance received
G4 Design approval	13–15	Technical Lead, client (UI)	Architecture + ADRs, ERD, prototype signed
G5 Ready to build	16–17	Technical Lead, PM	Plan, protected repo, CI skeleton
G6 Release candidate	22	QA Lead, Technical Lead	No open critical/high defects; coverage targets
G7 UAT sign-off	23	Client sponsor	Signed UAT certificate
G8 Go-live	25	Department Head, client	Rehearsal passed, backup verified, rollback plan
G9 Handover	28	Client, Department Head	Handover pack and certificate

TEMPLATE CODES

Appendices C–K of this handbook contain the full text of the core templates (BRD, FRD, SOW, DB, QA, UAT, deployment, handover, costing). Codes **T01–T19** are template IDs reserved for the lighter working forms (scorecards, registers, minutes); they are **produced in Modules M1–M6** and are not yet written — until then, use the checklist in the stage's Tasks row.

Phase A — Discover & Define (Stages 1–12)

Stage 01 — Lead qualification

Owner	A: Sales/Department Head · R: Business Analyst
Tasks	• Capture lead source, contact, sector, problem statement, rough budget and timeline. • Score fit: strategic fit, budget realism, decision-maker access, technical feasibility, risk (1–5 each). • Reject or redirect unsuitable leads (illegal, no budget, impossible deadline, outside capability). • Book a discovery meeting only for leads scoring ≥ 18/25.
Inputs	Enquiry, website form, referral note
Outputs	Lead scorecard, qualification note, Go/No-Go decision
Templates	T01 Lead qualification form
Review	Department Head reviews every lead scored 15–20
Approval gate	**G1 Qualification** — Department Head signs Go/No-Go

Stage 02 — Client discovery

Owner	A: Project Manager · R: Business Analyst
Tasks	• Prepare agenda and question bank; send pre-read. • Run discovery workshop(s) with owner and department heads. • Record goals, pains, current tools, volumes, locations, deadlines, constraints, budget bands. • Send minutes within 24 h and get written acknowledgement.
Inputs	Qualified lead, client documents (forms, reports, Excel sheets)
Outputs	Discovery notes, problem statement, success measures, initial risks
Templates	T02 Discovery questionnaire & minutes
Review	Second BA reads notes for gaps
Approval gate	Client acknowledges minutes by e-mail

Stage 03 — Business process study

Owner	A: Business Analyst · R: Business Analyst, Technical Lead (advice)
Tasks	• Shadow users doing real work; collect real forms, spreadsheets, invoices, reports. • List processes, triggers, steps, volumes, exceptions and pain points. • Capture the business rules and calculations with worked examples from real data.
Inputs	Discovery notes, sample documents
Outputs	Process inventory, business-rule catalogue, sample data set
Templates	T03 Process inventory
Review	Peer BA review
Approval gate	Client department head confirms process inventory

Stage 04 — Stakeholder identification

Owner	A: Project Manager · R: Business Analyst
Tasks	• Identify sponsor, decision-maker, process owners, end users, IT contact, finance approver, external parties. • Build a power/interest map and communication plan; agree one single point of contact (SPOC) on each side. • Agree decision rights: who can approve requirements, changes and UAT.
Inputs	Org chart, discovery notes
Outputs	Stakeholder register, RACI for the project, communication plan
Templates	T04 Stakeholder register
Review	Project Manager review
Approval gate	Sponsor confirms decision-makers in writing

Stage 05 — AS-IS process documentation

Owner	A: Business Analyst · R: Business Analyst
Tasks	• Draw each in-scope process end-to-end (BPMN swimlane) exactly as performed today. • Annotate delays, duplicate entry, errors, controls and volumes. • Validate with the people who do the work, not only managers.
Inputs	Process inventory, shadowing notes
Outputs	AS-IS process maps with pain points and metrics
Templates	T05 BPMN/flow template
Review	Process owner walk-through; peer review
Approval gate	Process owners sign each AS-IS map as accurate

Stage 06 — TO-BE process design

Owner	A: Business Analyst · R: BA, Technical Lead
Tasks	• Design the improved flow: remove duplication, add controls and approvals, define system vs manual steps. • List automation points, new roles, and changes to current practice. • Estimate the benefit (time saved, errors avoided) so the client sees value.
Inputs	Approved AS-IS maps
Outputs	TO-BE maps, gap analysis, benefit estimate, change-impact list
Templates	T05 BPMN/flow template, T06 Gap analysis
Review	Technical Lead checks feasibility; BA peer review
Approval gate	Sponsor approves TO-BE direction

Stage 07 — Functional requirement gathering

Owner	A: Business Analyst · R: Business Analyst
Tasks	• Write numbered requirements (FR-001…) per module, each atomic, testable and traceable to a TO-BE step. • Write user stories with Given/When/Then acceptance criteria and priority (MoSCoW). • Record business rules, validations, calculations, reports and notifications.
Inputs	TO-BE maps, rule catalogue
Outputs	Functional Requirements Specification (FRS), user-story backlog, traceability matrix v0
Templates	Appendix C/D (BRD & FRD)
Review	BA peer + Technical Lead review for testability
Approval gate	Client SPOC accepts FRS draft

Stage 08 — Non-functional requirements

Owner	A: Technical Lead · R: Technical Lead, BA
Tasks	• Quantify performance (users, response time), availability, security, audit, retention, backup RPO/RTO, browser/device support, accessibility (WCAG 2.2 AA target), localisation. • Record data volume growth and compliance-readiness needs. • State every NFR with a number and a way to measure it.
Inputs	Discovery notes, volumes
Outputs	NFR specification with measurable targets
Templates	T07 NFR catalogue
Review	Technical Lead + Security reviewer
Approval gate	Client signs NFRs as part of freeze

Stage 09 — Input/output specification

Owner	A: Business Analyst · R: BA, Backend Developer
Tasks	• List every screen input field (name, type, length, mandatory, validation, source). • List every report/document/export/interface output (layout, filters, formulas, sample). • Define import/export files and third-party interface contracts.
Inputs	FRS, sample documents
Outputs	I/O specification, field dictionary, report catalogue, interface list
Templates	T08 Field dictionary & report spec
Review	Developer confirms feasibility; client confirms sample outputs
Approval gate	Client signs sample outputs

Stage 10 — User roles & permissions

Owner	A: Technical Lead · R: BA, Backend Developer
Tasks	• Define roles, permission matrix (role × action × data scope), separation of duties (maker ≠ checker), admin and support access rules. • Define authentication method (SSO, e-mail+MFA) and session rules. • Define audit events to record.
Inputs	FRS, stakeholder register
Outputs	Role-permission matrix, audit-event list, auth approach
Templates	T09 Permission matrix
Review	Security review; client process owners
Approval gate	Client approves matrix

Stage 11 — Requirements freeze

Owner	A: Project Manager · R: BA, Technical Lead
Tasks	• Walk the client through FRS, NFR, I/O and permission matrix line by line. • Resolve open issues; assign baseline version (e.g. FRS v1.0). • Publish the change-request procedure; from here, changes follow Ch 8 pricing.
Inputs	All Phase A documents
Outputs	Signed baseline: BRD, FRS, NFR, I/O spec, permission matrix, traceability matrix
Templates	T10 Requirements sign-off
Review	Department Head + Technical Lead sign-off
Approval gate	**G2 Requirements freeze** — client sponsor and Department Head sign

Stage 12 — SOW & commercial approval

Owner	A: Department Head · R: Project Manager, BA
Tasks	• Convert baseline to modules, estimate with the costing model (Ch 8), add risk contingency. • Draft SOW: scope, deliverables, milestones, acceptance, assumptions, exclusions, payments, warranty, support, IP, confidentiality, termination. • Legal/finance review; negotiate; obtain signed SOW and advance payment.
Inputs	Frozen requirements, costing model
Outputs	Signed SOW, quote, payment schedule, project charter
Templates	Appendix E (SOW), costing spec
Review	Finance (margin ≥ target), Legal, Technical Lead (estimate)
Approval gate	**G3 Commercial approval** — Department Head signs; client signs SOW

Phase B — Design & Plan (Stages 13–17)

Stage 13 — Software architecture

Owner	A: Technical Lead · R: Technical Lead, Backend Developer
Tasks	• Select style (default: modular monolith), modules and boundaries; apply the stack matrix (Ch 4). • Draw C4 context/container/component diagrams. • Write ADRs for each significant decision; define cross-cutting concerns (auth, logging, validation, error model, config, caching).
Inputs	Frozen requirements
Outputs	Architecture document, C4 diagrams, ADRs, tech-stack decision
Templates	T11 Architecture & ADR templates
Review	Peer architect/Technical Lead from another project + Security
Approval gate	**G4 Design approval** (with stages 14–15)

Stage 14 — Database architecture

Owner	A: Database Engineer · R: Database Engineer
Tasks	• Model entities (ERD) in 3NF; define keys, constraints, indexes, naming, audit columns and soft-delete policy. • Plan migrations, seed/reference data, partitioning/archival, backup and restore strategy with RPO/RTO. • Where multi-tenant is required: choose isolation model and write tenant-isolation tests first.
Inputs	FRS, field dictionary, NFRs
Outputs	ERD, data dictionary, DDL draft, migration plan, backup plan
Templates	Appendix F (DB checklist)
Review	Second Database Engineer/Technical Lead
Approval gate	Included in G4

Stage 15 — UI/UX design

Owner	A: Frontend Developer (UX lead) · R: Frontend Developer, BA
Tasks	• Create navigation map and wireframes per role; apply design system and accessibility rules. • Prototype key flows (clickable); usability-test with 3 real users. • Define empty/error/loading states and print layouts.
Inputs	FRS, I/O spec, brand guide
Outputs	Wireframes, clickable prototype, UI standards, accessibility notes
Templates	T12 Wireframe & UI checklist
Review	BA validates against stories; client approves prototype
Approval gate	Included in G4 — client signs prototype

Stage 16 — Development planning

Owner	A: Project Manager · R: Technical Lead, PM
Tasks	• Break work into epics → stories → tasks; estimate in ideal days with a 3-point range; order by risk and value. • Plan 2-week sprints, definition of ready/done, environments, risks, resource plan and release schedule. • Agree reporting rhythm: weekly status, fortnightly demo.
Inputs	Architecture, backlog
Outputs	Release plan, sprint-1 backlog, RAID log, resource plan
Templates	T13 Project plan & RAID log
Review	Technical Lead and Department Head
Approval gate	**G5 Ready to build**

Stage 17 — Repository creation

Owner	A: Technical Lead · R: Technical Lead
Tasks	• Create the repository from the department template (Ch 6); protect `main`; enable required reviews and status checks. • Configure secret scanning, dependency alerts, CODEOWNERS, issue/PR templates, labels. • Add CI skeleton (build + test + lint) and local dev `docker compose` environment.
Inputs	Architecture, standards
Outputs	Repository, CI skeleton, branch rules, developer README
Templates	Repo template, PR template
Review	Another engineer clones and runs it in < 30 min
Approval gate	Included in G5

Phase C — Build (Stages 18–21)

Stage 18 — Module-wise development

Owner	A: Technical Lead · R: Developers (Backend/Frontend/Junior)
Tasks	• Develop module by module in vertical slices; each story ends with tests, docs and a demo. • Follow standards (Ch 7): layers, validation, transactions, audit, RBAC, error model. • Every PR: small, linked to story, reviewed by someone else, CI green before merge.
Inputs	Sprint backlog, architecture, UI prototype
Outputs	Working, tested increments each sprint; updated docs; release notes
Templates	T14 PR checklist, Definition of Done
Review	Mandatory PR review; Technical Lead reviews architecture-affecting PRs
Approval gate	Sprint review accepted by Product Owner (client SPOC)

Stage 19 — API & integration development

Owner	A: Technical Lead · R: Integration Engineer
Tasks	• Design APIs contract-first (OpenAPI); version them. • Build adapters for accounting, payment, SMS/e-mail, biometric, government portals; use idempotency keys, retries with back-off, timeouts, dead-letter queues. • Stub third parties in tests; log every call with correlation ID, never secrets.
Inputs	Interface list, vendor API docs
Outputs	API specs, adapters, integration test suite, runbook for failures
Templates	T15 Integration design sheet
Review	Technical Lead; vendor sandbox test evidence
Approval gate	Integration test pass report

Stage 20 — Database migration (schema)

Owner	A: Database Engineer · R: Database Engineer, Backend Developer
Tasks	• Write forward-only versioned migrations; test on a production-sized copy; measure lock time. • Use expand → migrate → contract for breaking changes; never edit an applied migration. • Script seeds for reference data; rehearse rollback/restore.
Inputs	DDL draft, ERD
Outputs	Migration scripts, seed data, rehearsal log
Templates	Appendix F (DB checklist)
Review	Database Engineer peer
Approval gate	Migration rehearsal signed by Technical Lead

Stage 21 — Security implementation

Owner	A: Technical Lead · R: All developers; Security reviewer
Tasks	• Implement authN/authZ, input validation, output encoding, secure file upload, secrets management, TLS, rate limiting, audit logging. • Run SAST, dependency, secret and container scans in CI; fix criticals before release. • Threat-model each module (STRIDE) and test against the OWASP ASVS/Top 10 checklist.
Inputs	Permission matrix, NFRs, threat model
Outputs	Security test report, scan results, hardening checklist
Templates	T16 Security checklist
Review	Independent reviewer (not the author)
Approval gate	No open critical/high findings without written risk acceptance

Phase D — Verify & Release (Stages 22–26)

Stage 22 — Testing

Owner	A: QA Engineer · R: QA, Developers
Tasks	• Execute unit, integration, system, regression, performance, security and accessibility tests; trace each FR to ≥ 1 test. • Log defects with severity, steps, evidence; retest fixes; maintain regression pack. • Report coverage, defect density and open defects daily in the final week.
Inputs	Build, FRS, test plan
Outputs	Test plan, cases, execution reports, defect log, release-candidate report
Templates	Appendix H (QA checklist)
Review	QA Lead sign-off; Technical Lead review
Approval gate	**G6 Release candidate** — zero open critical/high defects

Stage 23 — User acceptance testing (UAT)

Owner	A: Project Manager · R: Client SPOC, BA, QA
Tasks	• Prepare UAT scripts from acceptance criteria with real sample data; train UAT testers. • Client executes scripts in staging; defects triaged within 24 h; retest. • Record formal acceptance, per module.
Inputs	Release candidate on staging, UAT scripts
Outputs	UAT results, sign-off certificate, accepted-defect list
Templates	Appendix I (UAT checklist)
Review	BA verifies each criterion; PM manages issues
Approval gate	**G7 UAT sign-off** — client sponsor signs

Stage 24 — Staging deployment

Owner	A: Technical Lead · R: Backend Developer, DevOps support
Tasks	• Deploy with the same pipeline and configuration shape as production; use masked/synthetic data. • Run smoke tests, performance baseline and restore drill; rehearse the production runbook. • Freeze the release candidate (tag, SBOM, image digest).
Inputs	Release candidate, runbook
Outputs	Staging environment, rehearsal log, release notes
Templates	Appendix J (Deployment checklist)
Review	Second engineer executes the runbook from the document only
Approval gate	Rehearsal passed

Stage 25 — Production deployment

Owner	A: Technical Lead · R: Backend Developer, Support Engineer
Tasks	• Announce change window; take verified backup; deploy via pipeline; run smoke tests; monitor 24–72 h hyper-care. • Execute rollback if exit criteria fail. • Record change, approver, time, result.
Inputs	Signed UAT, rehearsed runbook, backup
Outputs	Live system, release record, monitoring in place, hyper-care log
Templates	Appendix J (Deployment checklist)
Review	Change approver ≠ deployer
Approval gate	**G8 Go-live** — Department Head + client sponsor approve

Stage 26 — Data migration (go-live load)

Owner	A: Database Engineer · R: DB Engineer, BA, Client data owner
Tasks	• Extract from legacy; cleanse and map; load in rehearsals (≥ 2 dry runs); reconcile counts, totals and samples. • Client data owner signs reconciliation before cut-over; keep a read-only legacy copy. • Run final delta load in the cut-over window.
Inputs	Mapping sheets, legacy extracts
Outputs	Migrated data, reconciliation report, exception list
Templates	T17 Data-migration plan & reconciliation
Review	Client data owner + DB Engineer sign
Approval gate	Reconciliation signed (record counts and financial totals match)

Phase E — Adopt & Sustain (Stages 27–29)

Stage 27 — Employee (end-user) training

Owner	A: Project Manager · R: BA, Support Engineer
Tasks	• Create role-based training plan, user manuals, quick guides, short videos and FAQs. • Train admins and users in small groups on staging with realistic data; test with a short quiz. • Appoint super-users inside the client.
Inputs	Final UI, manuals
Outputs	Training materials, attendance, assessment results, super-user list
Templates	T18 Training plan & attendance
Review	Client HR/department head confirms attendance
Approval gate	≥ 90 % of users trained before go-live

Stage 28 — Client handover

Owner	A: Project Manager · R: Technical Lead, Support Engineer
Tasks	• Hand over source/IP per SOW, credentials via secure vault, architecture, runbooks, backup/restore guide, API docs, user manuals. • Walk the client IT through operations; transfer ownership of accounts and domains. • Close the project: lessons learned, final invoice, satisfaction survey.
Inputs	All deliverables
Outputs	Handover pack, signed acceptance, support contact matrix
Templates	Appendix K (Handover checklist)
Review	Technical Lead and Department Head
Approval gate	**G9 Handover** — client signs handover certificate

Stage 29 — Maintenance & upgrades

Owner	A: Support Lead · R: Support Engineer, Developers
Tasks	• Operate the support desk with SLA tiers (P1–P4), monitoring, monthly patching, backup verification and monthly report. • Triage requests: bug (warranty) / change request (priced) / enhancement (roadmap). • Plan upgrades: runtime/framework, database, dependencies and annual architecture health check; propose roadmap.
Inputs	Support contract, monitoring, backlog
Outputs	Monthly report, patched system, upgrade roadmap, renewal proposal
Templates	T19 SLA & monthly report
Review	Technical Lead quarterly review
Approval gate	Client approves each upgrade window

Official references for this chapter

- C4 model — `https://c4model.com/`
- OpenAPI Specification — `https://spec.openapis.org/oas/latest.html`
- OWASP ASVS — `https://owasp.org/www-project-application-security-verification-standard/`
- Martin Fowler — articles & catalogue — `https://martinfowler.com/`
- GitHub Docs — `https://docs.github.com/`

6. Enterprise Project Folder Structure

Every client repository starts from the same template so any engineer can find anything in seconds. The structure follows **Clean/Onion layering** (domain at the centre, infrastructure at the edge) inside a **modular monolith** by default.

```
TREE
enterprise-software-project/
├── README.md                    # what it is, how to run in <30 min, links to docs
├── CONTRIBUTING.md              # branching, commits, PR rules, Definition of Done
├── CODEOWNERS                   # mandatory reviewers per path
├── docs/
│   ├── business-requirements/   # BRD, stakeholder register, AS-IS / TO-BE maps
│   ├── functional-requirements/ # FRS, user stories, field dictionary, traceability
│   ├── workflows/               # BPMN / state diagrams, approval matrices
│   ├── architecture/            # C4 diagrams, ADRs (adr-0001-*.md), NFRs
│   ├── database/                # ERD, data dictionary, migration & backup plan
│   ├── api/                     # OpenAPI files, integration contracts
│   └── user-manuals/            # role-based guides, training material, FAQ
├── src/
│   ├── frontend/                # React/Angular app (or WPF shell)
│   ├── backend/                 # API host: controllers/endpoints, DI, middleware
│   ├── domain/                  # entities, value objects, domain rules (no I/O)
│   ├── application/             # use cases, commands/queries, validators, interfaces
│   ├── infrastructure/          # EF Core/SQL repositories, e-mail, storage, clock
│   └── integrations/            # adapters to accounting, payment, SMS, government APIs
├── database/
│   ├── migrations/              # versioned, forward-only scripts
│   ├── seeds/                   # reference data (roles, tax codes, statuses)
│   └── scripts/                 # backup, restore, reporting views, maintenance
├── tests/
│   ├── unit/                    # fast tests: domain + application
│   ├── integration/             # real DB (container) + API tests
│   ├── system/                  # end-to-end, performance, security checks
│   └── acceptance/              # UAT scripts mapped to acceptance criteria
├── deployment/                  # Dockerfiles, compose, IaC, runbooks, release notes
├── security/                    # threat models, security checklist, scan config, policies
├── monitoring/                  # dashboards, alert rules, health-check definitions, SLOs
└── .github/
    └── workflows/               # ci.yml, security.yml, deploy-staging.yml, deploy-prod.yml
```

What each directory is for

Directory	Purpose	Rules
`docs/business-requirements`	The *why*: BRD, stakeholders, AS-IS/TO-BE.	Signed versions are immutable; changes create a new version.
`docs/functional-requirements`	The *what*: FRS, stories, field dictionary, traceability matrix.	Every FR ID appears in at least one test.
`docs/workflows`	Approval flows and state machines.	Diagrams stored as source (`.bpmn` , `.drawio`) plus exported PNG.

Directory	Purpose	Rules
`docs/architecture`	C4 diagrams, ADRs, NFRs.	One ADR per significant decision; never delete, supersede.
`docs/database`	ERD, data dictionary, backup/restore plan.	Updated in the same PR as any migration.
`docs/api`	OpenAPI specs and integration contracts.	Generated or validated in CI; breaking changes need version bump.
`docs/user-manuals`	End-user and admin guides.	Written for roles, with screenshots from staging.
`src/domain`	Entities, value objects, domain services and rules.	**No** references to database, HTTP or UI frameworks.
`src/application`	Use cases, DTOs, validators, ports (interfaces).	Defines transactions boundaries; depends only on domain.
`src/infrastructure`	Implementations of ports: EF Core, storage, e-mail, clock.	Replaceable; contains no business rules.
`src/backend`	Composition root: endpoints, auth, DI, logging, health checks.	Thin — maps HTTP to use cases.
`src/integrations`	Anti-corruption adapters to external systems.	One folder per external system; contract tests with recorded samples.
`src/frontend`	UI application.	Talks to the backend only through the generated API client.
`database/migrations`	Versioned schema changes (`V0001__create_party.sql`).	Applied migrations are never edited.
`database/seeds`	Reference data only; no personal data.	Idempotent scripts.
`database/scripts`	Backup/restore/maintenance scripts.	Dry-run flag and confirmation prompt on destructive scripts.
`tests/*`	Test pyramid layers.	Unit tests run < 2 minutes; integration tests use disposable containers.
`deployment`	Dockerfiles, compose, IaC, runbooks.	No secrets; configuration via environment/secret store.
`security`	Threat models, checklist, scan configs, risk register.	Reviewed every release.
`monitoring`	Dashboards-as-code, alert rules, SLOs.	Every alert has an owner and a runbook link.
`.github/workflows`	CI/CD pipelines.	Pinned action versions; least-privilege tokens.

Adapting the structure to the chosen architecture

Architecture	Adaptation
Modular monolith (default)	Inside `src/`, give each business module its own folder per layer or its own project: `src/modules/inventory/{domain,application,infrastructure}`. Modules talk through public interfaces or in-process events only; an architecture test fails the build on forbidden references.
Microservices (justified by ADR)	One repository per service **or** a mono-repo with `services/<name>/` each having its own `src/`, `database/`, `deployment/`. Add `contracts/` (OpenAPI/AsyncAPI) and `platform/` (gateway, broker, observability). Each service owns its database — no shared tables.
WPF desktop	`src/frontend` becomes `src/desktop-wpf` (MVVM). Keep `domain` / `application` UI-free class libraries so the same rules can serve a future web API. `deployment/` holds the installer (MSIX/WiX) and update channel.
React + FastAPI	`src/backend` is the Python package (`app/api`, `app/services`, `app/models`, `alembic/`); `domain/application/infrastructure` map to `app/domain`, `app/services`, `app/repositories`.
Legacy modernisation	Add `legacy/` (read-only snapshots, schema dumps) and `docs/migration/` (mapping sheets, reconciliation reports, strangler routing table).

DEFINITION OF A READY REPOSITORY (STAGE 17)

A new engineer can clone, run `docker compose up`, run the tests and open the app in **under 30 minutes** using only the README.

Official references for this chapter

- C4 model — `https://c4model.com/`
- GitHub Docs — `https://docs.github.com/`
- Docker Docs — `https://docs.docker.com/`
- Martin Fowler — articles & catalogue — `https://martinfowler.com/`

7. Mandatory Enterprise Engineering Standards

These sixteen standards are enforced in code review and CI. Each has a **rule**, an **implementation pattern** and a **verification**. A standard may be waived only by the Technical Lead, in writing, with a risk note in the ADR log.

#	Standard	Rule	How we verify
1	**Modular architecture**	Business capabilities are modules with explicit public APIs.	Architecture tests block forbidden dependencies.
2	**Separation of concerns**	UI, API, application, domain and persistence are separate layers; no business logic in controllers or SQL triggers.	Code review checklist; layering tests.
3	**Reusable business components**	Shared concerns (numbering series, tax, approval, attachments, audit) are built once in a shared kernel.	No duplicate logic in PR review.
4	**Database integrity**	PK/FK/UNIQUE/CHECK/NOT NULL on every table; money as `numeric/decimal`; UTC timestamps.	DB checklist (Appendix F).
5	**Transaction management**	One use case = one transaction; explicit boundaries; optimistic concurrency on editable records.	Rollback and concurrency tests.
6	**Role-based access control**	Deny by default; permission checks server-side at endpoint **and** object level.	Permission-matrix test per role.
7	**Audit logs**	Who/what/when/old/new for every create/update/delete/approve; append-only.	Audit tests; DB account cannot update/delete audit rows.
8	**Multi-tenant isolation**	Only if the client requires it. Tenant ID on every row, enforced centrally (global filter + DB row-level security where available).	Mandatory cross-tenant tests (below).
9	**Data validation**	Validate at the edge (shape/type/range) **and** in the domain (business rules) **and** in the DB (constraints).	Negative tests for each rule.
10	**Secure file handling**	Whitelist type by content, size limit, random storage names, outside web root, malware scan hook, signed download links.	Upload abuse tests.
11	**Backup & restore**	Automated backups, off-site copy, encrypted, **restore drill each quarter**; RPO/RTO documented.	Drill report.
12	**API security**	TLS only; authN on every endpoint; rate limits; input limits; CORS allow-list; no sensitive data in URLs; OWASP API Top 10 review.	ZAP/automated scan + manual checklist.
13	**Performance**	Budgets in NFRs (e.g. p95 < 500 ms for list screens); pagination everywhere; indexes justified by query plans; no N+1.	Load test + query-plan review.

#	Standard	Rule	How we verify
14	**Accessibility**	WCAG 2.2 AA target: keyboard operable, labels, contrast, focus order, error text.	axe/Lighthouse + manual keyboard test.
15	**Versioning**	Semantic versioning for releases; API versions; DB migration versions; documented deprecations.	Release checklist.
16	**Documentation**	README, ADRs, API docs, runbooks, user manuals updated in the same PR as the change.	PR template checkbox; docs build in CI.

Implementation patterns

Transaction + audit in one unit of work (standards 5 and 7)

CSHARP

```
public sealed class ApproveInvoiceHandler(AppDbContext db, ICurrentUser user, IClock clock)
{
    public async Task Handle(Guid invoiceId, CancellationToken ct)
    {
        await using var tx = await db.Database.BeginTransactionAsync(ct);
        var inv = await db.Invoices.SingleAsync(i => i.Id == invoiceId, ct);      // tenant filter
applied globally
        if (inv.CreatedBy == user.Id) throw new DomainException("Maker cannot approve own invoice.");
        var before = inv.Status;
        inv.Approve(user.Id, clock.UtcNow);                                       // domain rule
inside entity
        db.Audit.Add(new AuditEntry("Invoice", inv.Id, "Approve", before, inv.Status, user.Id,
clock.UtcNow));
        await db.SaveChangesAsync(ct);                                            // RowVersion
concurrency token checked
        await tx.CommitAsync(ct);
    }
}
```

Multi-tenant isolation (standard 8) — only with explicit requirement

> **NEVER DESIGN MULTI-TENANCY WITHOUT EXPLICIT TENANT-ISOLATION REQUIREMENTS AND TESTS**
>
> Before any tenant column is added, the SOW/FRS must state: how many tenants, isolation level (shared schema / schema-per-tenant / database-per-tenant), who may see cross-tenant data (nobody by default), and what happens on tenant offboarding. If these are missing, **stop and ask** — build single-tenant.

CSHARP / SQL

```
// EF Core global query filter — every tenant-owned entity implements ITenantOwned
modelBuilder.Entity<Invoice>().HasQueryFilter(e => e.TenantId == _tenant.Id);

// PostgreSQL defence in depth (row-level security)
alter table invoice enable row level security;
create policy tenant_isolation on invoice
  using (tenant_id = current_setting('app.tenant_id')::uuid);
```

CSHARP

```
// Mandatory isolation tests (xUnit) — a build with any failing test cannot be released
[Fact] public async Task TenantA_cannot_read_TenantB_invoice() { /* seed B, query as A -> empty/404
*/ }
[Fact] public async Task TenantA_cannot_update_TenantB_invoice_by_guessing_id() { /* expect 404/403,
row unchanged */ }
[Fact] public async Task Reports_and_exports_filter_by_tenant() { }
[Fact] public async Task Background_jobs_run_with_explicit_tenant_context() { }
[Fact] public async Task Missing_tenant_context_fails_closed() { /* no tenant -> exception, not 'all
rows' */ }
```

Secure file upload (standard 10)

CSHARP

```
var allowed = new Dictionary<string, byte[]> { [".pdf"] = "%PDF"u8.ToArray(), [".png"] =
[0x89,0x50,0x4E,0x47] };
if (file.Length is 0 or > 10_000_000) return Results.BadRequest("Invalid size");
var ext = Path.GetExtension(file.FileName).ToLowerInvariant();
if (!allowed.TryGetValue(ext, out var magic)) return Results.BadRequest("Type not allowed");
await using var s = file.OpenReadStream(); var head = new byte[magic.Length]; await
s.ReadExactlyAsync(head);
if (!head.SequenceEqual(magic)) return Results.BadRequest("Content mismatch");
var stored = $"{Guid.NewGuid():N}{ext}";            // never trust the client file name
await storage.SaveAsync(stored, s, ct);              // outside web root; scan hook; DB keeps
metadata
```

Validation at three levels (standard 9)

Level	Example	Why
Edge (API)	Required, length ≤ 100, valid e-mail, amount > 0	Fail fast with 400 and field-level messages.
Domain	Credit limit not exceeded; cannot ship more than ordered	Rules hold no matter which client calls.
Database	`CHECK (qty >= 0)`, `UNIQUE (tenant_id, invoice_no)`	Last line of defence against bugs and direct SQL.

STRUCTURAL/REGULATORY HONESTY

Standards reduce risk; they do not guarantee security or legal compliance. Security reports carry the line “Independent engineer/auditor review required before production reliance.”

Official references for this chapter

- OWASP Top 10 — `https://owasp.org/www-project-top-ten/`
- OWASP API Security Top 10 — `https://owasp.org/www-project-api-security/`
- OWASP ASVS — `https://owasp.org/www-project-application-security-verification-standard/`
- PostgreSQL documentation — `https://www.postgresql.org/docs/`
- ASP.NET Core documentation — `https://learn.microsoft.com/aspnet/core/`
- WCAG 2.2 — `https://www.w3.org/TR/WCAG22/`

8. Costing and Commercial Model

Price = **cost to deliver ÷ (1 − target gross margin)**, built bottom-up from effort, never from what the client "can afford". All figures below are **worked illustrations with internal assumptions** (INR, ex-GST); recalibrate rates from timesheets each quarter.

VENDOR PRICING — VERIFY BEFORE QUOTING

Hosting, database, licence and SaaS prices change. The hosting and licence lines below are **allowances**, not vendor quotes. Before any quotation, price the real services in the official calculators — AWS Pricing Calculator, Azure Pricing Calculator, Google Cloud Pricing Calculator, GitHub Plans & pricing, Microsoft SQL Server pricing — and record the date, region, SKU and screenshot in the quote file. GST/tax treatment must be confirmed with the finance owner and official tax portal.

Cost components (all 17 must appear in every estimate)

Component	How it is estimated	Cost basis
Discovery	8% of development days (min 3 days)	BA day rate
Business analysis	12% of dev days	BA day rate
Architecture	6% of dev days	Technical Lead day rate
UI/UX	12% of dev days	UX/Frontend day rate
Development	Σ module days (3-point estimate: (O + 4M + P) / 6)	Developer day rate
Database	10% of dev days	DB Engineer day rate
Integration	Per interface: simple 3 d, medium 6 d, complex 12 d	Integration Engineer day rate
Migration	Per source: 2 d + 1 d per 10 tables + cleansing allowance	DB Engineer day rate
QA	30% of dev days	QA day rate
Deployment	5% of dev days	Tech Lead/DevOps day rate
Training	5% of dev days + client sessions	BA/Support day rate
Documentation	6% of dev days	BA/dev day rate
Licences	Itemised from vendor quote; client-owned where possible	Pass-through + handling
Hosting	Staging + production from official calculators × months	Pass-through + management fee

Component	How it is estimated	Cost basis
Support & maintenance	% of project price per year (see below)	Support Engineer + developer time
Future upgrades	Reserved annual block for framework/DB/security upgrades	Developer time
Project management	12% of all other effort days	PM day rate (₹6,000)

Assumed internal cost rates (INR / person-day)

Activity	Cost / day	Billing rate card at 40 % margin
Discovery	₹4,000	₹6,667
Business analysis	₹4,000	₹6,667
Architecture	₹8,000	₹13,333
UI/UX	₹4,000	₹6,667
Development	₹4,500	₹7,500
Database	₹5,000	₹8,333
Integration	₹5,000	₹8,333
Migration	₹5,000	₹8,333
QA	₹3,000	₹5,000
Deployment	₹5,500	₹9,167
Training	₹3,500	₹5,833
Documentation	₹3,500	₹5,833
Project management	₹6,000	₹10,000

FORMULAS

1. Labour cost = Σ (days × cost/day). Contingency = labour × risk % (small 10 %, medium 15 %, enterprise 20 %).
2. Direct cost = labour + contingency + licences + build-period hosting. **Price = direct cost ÷ (1 − 40%).**
3. Gross margin % = (price − direct cost) ÷ price. Discounts reduce margin 1:1 — never discount without Department Head approval.
4. Break-even projects per year = 12 × monthly non-billable overhead ÷ gross margin per project.
5. Annual support = support % × project price; annual hosting = monthly hosting × 12 × 1.15 (management fee).

Example — Small: Single-clinic appointment & billing system

Team 3 · indicative build **2 month(s)** · 79.7 person-days · Free/open-source stack; SMS/WhatsApp provider charges billed to client.

Cost line	Days	Rate / day	Cost
Development	33.0	₹4,500	₹1,48,500
Integration	4.0	₹5,000	₹20,000
Migration	3.0	₹5,000	₹15,000
Discovery	2.6	₹4,000	₹10,400
Business analysis	4.0	₹4,000	₹16,000
Architecture	2.0	₹8,000	₹16,000
UI/UX	4.0	₹4,000	₹16,000
Database	3.3	₹5,000	₹16,500
QA	9.9	₹3,000	₹29,700
Deployment	1.7	₹5,500	₹9,350
Training	1.7	₹3,500	₹5,950
Documentation	2.0	₹3,500	₹7,000
Project management	8.5	₹6,000	₹51,000
Labour subtotal	**79.7**		**₹3,61,400**
Contingency (10%)			₹36,140
Licences (allowance)			₹0

Cost line	Days	Rate / day	Cost
Build-period hosting (2 × ₹6,000)			₹12,000
Direct cost			**₹4,09,540**
Project price (÷ 0.60)			**₹6,82,567**
Gross margin			₹2,73,027 (40%)

Module-wise price

Module	Dev days	Price
Patient registration	6	₹1,08,800
Appointments & slots	8	₹1,45,067
Billing & receipts	10	₹1,81,333
Reports	4	₹72,533
Users, roles & audit	5	₹90,667
Integration + data migration (incl. contingency & margin)	7	₹64,167
Licences + build hosting (with margin)	—	₹20,000
Total = project price	**33**	**₹6,82,567**

Recurring costs and year-1 view

Item	Basis	Per year
Annual support	15% of project price (SLA tiers P1–P4, monthly report)	₹1,02,385
Hosting & monitoring	₹6,000/month × 12 + 15 % management fee (verify with calculators)	₹82,800
Planned upgrades	8% of project price reserved block	₹54,605
Total recurring		**₹2,39,790**

Break-even: with assumed non-billable overhead of ₹3,00,000/month, the department must deliver **13.2 projects of this size per year** to cover overhead from project margin alone (recurring-support margin is extra). Blended cost/day ₹4,535; blended rate-card day ₹7,558.

Example — Medium: Distributor inventory + billing + CRM with Tally integration

Team 5 · indicative build **4 month(s)** · 292.1 person-days · Managed PostgreSQL + app hosting; allowance for client-specific licences.

Cost line	Days	Rate / day	Cost
Development	123.0	₹4,500	₹5,53,500
Integration	12.0	₹5,000	₹60,000
Migration	10.0	₹5,000	₹50,000
Discovery	9.8	₹4,000	₹39,200
Business analysis	14.8	₹4,000	₹59,200
Architecture	7.4	₹8,000	₹59,200
UI/UX	14.8	₹4,000	₹59,200
Database	12.3	₹5,000	₹61,500
QA	36.9	₹3,000	₹1,10,700
Deployment	6.2	₹5,500	₹34,100
Training	6.2	₹3,500	₹21,700
Documentation	7.4	₹3,500	₹25,900
Project management	31.3	₹6,000	₹1,87,800
Labour subtotal	**292.1**		**₹13,22,000**
Contingency (15%)			₹1,98,300
Licences (allowance)			₹25,000
Build-period hosting (4 × ₹20,000)			₹80,000
Direct cost			**₹16,25,300**
Project price (÷ 0.60)			**₹27,08,833**
Gross margin			₹10,83,533 (40%)

Module-wise price

Module	Dev days	Price
Masters, users & roles	12	₹2,26,634

Module	Dev days	Price
Inventory & warehouses	25	₹4,72,154
Sales & billing	28	₹5,28,813
Purchase & GRN	15	₹2,83,293
CRM (leads, pipeline)	18	₹3,39,951
Approval workflow	10	₹1,88,862
Reports & dashboards	15	₹2,83,293
Integration + data migration (incl. contingency & margin)	22	₹2,10,833
Licences + build hosting (with margin)	—	₹1,75,000
Total = project price	**123**	**₹27,08,833**

Recurring costs and year-1 view

Item	Basis	Per year
Annual support	18% of project price (SLA tiers P1–P4, monthly report)	₹4,87,590
Hosting & monitoring	₹20,000/month × 12 + 15 % management fee (verify with calculators)	₹2,76,000
Planned upgrades	10% of project price reserved block	₹2,70,883
Total recurring		**₹10,34,473**

Break-even: with assumed non-billable overhead of ₹3,00,000/month, the department must deliver **3.3 projects of this size per year** to cover overhead from project margin alone (recurring-support margin is extra). Blended cost/day ₹4,526; blended rate-card day ₹7,543.

Example — Enterprise: Multi-branch hospital group management system

Team 9 · indicative build **6 month(s)** · 904.5 person-days · HA managed database, monitoring, backups; allowance for third-party tools.

Cost line	Days	Rate / day	Cost
Development	375.0	₹4,500	₹16,87,500
Integration	45.0	₹5,000	₹2,25,000
Migration	35.0	₹5,000	₹1,75,000
Discovery	30.0	₹4,000	₹1,20,000

Cost line	Days	Rate / day	Cost
Business analysis	45.0	₹4,000	₹1,80,000
Architecture	22.5	₹8,000	₹1,80,000
UI/UX	45.0	₹4,000	₹1,80,000
Database	37.5	₹5,000	₹1,87,500
QA	112.5	₹3,000	₹3,37,500
Deployment	18.8	₹5,500	₹1,03,400
Training	18.8	₹3,500	₹65,800
Documentation	22.5	₹3,500	₹78,750
Project management	96.9	₹6,000	₹5,81,400
Labour subtotal	**904.5**		**₹41,01,850**
Contingency (20%)			₹8,20,370
Licences (allowance)			₹2,00,000
Build-period hosting (6 × ₹85,000)			₹5,10,000
Direct cost			**₹56,32,220**
Project price (÷ 0.60)			**₹93,87,033**
Gross margin			₹37,54,813 (40%)

Module-wise price

Module	Dev days	Price
Registration & EMR core	30	₹5,92,296
Appointments	25	₹4,93,580
OPD/IPD	60	₹11,84,592
Billing & insurance	45	₹8,88,444
Pharmacy	40	₹7,89,728
Laboratory	35	₹6,91,012
Inventory & purchase	30	₹5,92,296
Workflow & approvals	25	₹4,93,580
Reports & BI	30	₹5,92,296

Module	Dev days	Price
Security, RBAC & audit	30	₹5,92,296
Multi-branch / tenant isolation	25	₹4,93,580
Integration + data migration (incl. contingency & margin)	80	₹8,00,000
Licences + build hosting (with margin)	—	₹11,83,333
Total = project price	**375**	**₹93,87,033**

Recurring costs and year-1 view

Item	Basis	Per year
Annual support	20% of project price (SLA tiers P1–P4, monthly report)	₹18,77,407
Hosting & monitoring	₹85,000/month × 12 + 15 % management fee (verify with calculators)	₹11,73,000
Planned upgrades	12% of project price reserved block	₹11,26,444
Total recurring		**₹41,76,851**

Break-even: with assumed non-billable overhead of ₹3,00,000/month, the department must deliver **1.0 projects of this size per year** to cover overhead from project margin alone (recurring-support margin is extra). Blended cost/day ₹4,535; blended rate-card day ₹7,558.

Commercial rules

Topic	Policy
Payment schedule (default)	30 % on SOW signature · 30 % on design approval (G4) · 30 % on UAT sign-off (G7) · 10 % on go-live + 30 days (G8/G9). Adjust only with Department Head approval.
Estimate validity	30 days; re-verify vendor prices at signature.
Warranty	Defects against signed requirements fixed free for 60–90 days after go-live; enhancements are change requests.
Discount authority	≤ 5 %: Project Manager · ≤ 10 %: Department Head · > 10 %: Director. Margin floor 30 %.
Fixed price vs T&M	Fixed price only after requirements freeze; T&M for discovery, legacy investigation and unclear scope.
Hosting/licences	Client-owned accounts preferred; if resold, show pass-through and management fee separately.
Taxes	Quotes are ex-tax; apply GST/other taxes per current law as confirmed by Finance.

Change-request pricing

1. Log the CR (ID, requester, description, reason, priority). The client SPOC must request in writing.
2. BA performs impact assessment (scope, schedule, cost, risk, tests, docs) — charged at **0.5 day BA rate** as a minimum fee, credited if the CR is approved.
3. Price = Σ(role days × rate card) + **20 % premium if raised after requirements freeze** (G2), 0 % if it replaces an equal-size in-scope item (scope swap).
4. Client approves in writing; Project Manager updates SOW schedule and baseline; only then is work started.

Worked example (Medium project): "Add a stock-transfer approval step between warehouses" — BA 1 d, Dev 4 d, QA 2 d, DB 0.5 d.

Line	Amount
Impact assessment (0.5 d BA at rate card)	₹3,333
Work: 1×BA + 4×Dev + 2×QA + 0.5×DB at rate card	₹50,833
Post-freeze premium (20 % of work)	₹10,167
Change-request price (ex-tax)	**₹64,333**

FIXING THE NUMBERS

These examples exist to teach the method. Do not quote a client from this chapter: build the real estimate with the **Costing Model Specification (Appendix L)**, get Technical Lead review of the days, and Finance review of margin.

Official references for this chapter

- AWS Pricing Calculator — `https://calculator.aws/`
- Azure Pricing Calculator — `https://azure.microsoft.com/pricing/calculator/`
- Google Cloud Pricing Calculator — `https://cloud.google.com/products/calculator`
- GitHub pricing — `https://github.com/pricing`

9. Employee Roles, Responsibility Matrix and Approvals

Ten roles, each with defined responsibilities, entry level and approval authority. In a startup one person may hold several roles, but **the author of work is never its approver**.

Role	Primary responsibility	Minimum level to work unsupervised	Key outputs
Business Analyst	Discovery, requirements, process maps, UAT preparation, change impact.	Levels 1–2 + Level 3 basics	BRD, FRS, stories, UAT scripts
Junior Software Developer	Implements assigned stories under review; writes tests.	Level 3 · pairs with a senior	PRs, unit tests
Backend Developer	APIs, domain logic, security, transactions, performance.	Levels 3, 4(basic), 6	Services, API docs, tests
Frontend Developer	UI, accessibility, UX prototypes, API client.	Level 3 (frontend track), 6	UI components, prototypes
Database Engineer	ERD, migrations, performance, backup/restore, data migration.	Levels 4–5	ERD, migrations, restore drills
Integration Engineer	Third-party and legacy interfaces, queues, idempotency.	Levels 3, 5	Adapters, contract tests
QA Engineer	Test strategy, automation, defect management, UAT support.	Levels 1–2, 3 basics, 6	Test plan, reports
Technical Lead	Architecture, standards, estimates, code/PR gatekeeping, mentoring.	All levels + 3 delivered projects	ADRs, reviews, approvals
Project Manager	Scope, plan, risks, budget, communication, gates, change control.	Levels 1–2 + PM training	Plans, status reports, change log
Support Engineer	Monitoring, tickets, patching, backups verification, monthly reports.	Levels 4 basics, 6	Tickets, reports, runbooks

Responsibility matrix (RACI)

R = does the work · **A** = accountable (one per row; signs) · **C** = consulted · **I** = informed. Roles: BA, JD (Junior Dev), BD (Backend), FD (Frontend), DB, IE (Integration), QA, TL (Technical Lead), PM, SE (Support). The Department Head additionally signs gates G1, G3, G8 and G9 and is accountable above the matrix.

Activity	BA	JD	BD	FD	DB	IE	QA	TL	PM	SE
Lead qualification	R							C	A	
Discovery & process study	R						I	C	A	
AS-IS / TO-BE design	R		I	I			I	C	A	
FR / NFR / I-O / permissions	R		C	C	C	C	C	A	I	
Requirements freeze	R		I	I	I	I	I	C	A	
SOW & costing	R		C					C	A	
Software architecture	C		R	C	C	C	I	A	I	
Database architecture	C		C		R	C	I	A	I	
UI/UX design	C			R			C	A	I	
Development planning	C	I	C	C	C	C	C	R	A	
Repository & CI setup		I	R	C			C	A	I	
Module development	C	R	R	R	C	C	I	A	I	
API & integrations	C		C			R	C	A	I	
Migrations & data migration	C		C		R		C	A	I	
Security implementation		R	R	R	C	C	C	A	I	I
Testing	C	C	C	C	C	C	R	A	I	
UAT	R						R	C	A	
Staging & production deploy			R		C		C	A	I	R
End-user training	R							C	A	R
Handover	R		R		C		C	C	A	R
Maintenance & upgrades		C	R	C	C	C	C	A	I	R

Approval structure and authority limits

Decision	Author / requester	Approver	Second reviewer / evidence
Pull request merge to `main`	Developer	Any peer with CODEOWNERS rights	CI green; tests added; docs updated
Architecture decision (ADR)	Technical Lead / Backend	Technical Lead of another project or Department Head	Alternatives + cost of ownership
Database migration to production	Database Engineer	Technical Lead	Rehearsal log; verified backup
Production deployment	Developer/Support	Technical Lead (client sponsor for go-live)	Change record; rollback plan

Decision	Author / requester	Approver	Second reviewer / evidence
Security exception / risk acceptance	Technical Lead	Department Head (client for client-owned risk)	Written risk statement with expiry date
Quotation / SOW	BA / PM	Department Head (Director above ₹50 lakh — adjust to policy)	Finance margin check; Legal review
Change request	Client SPOC	PM (≤ 5 % of project price) · Department Head (> 5 %)	Impact assessment
Employee level pass	Employee	Technical Lead / BA lead for the track	Quiz + demo + capstone review
Release of client data / access	Support Engineer	Client data owner + Technical Lead	Ticket with authorisation

SEGREGATION OF DUTIES

The developer of a change cannot approve its PR; the person who deploys cannot be the sole approver of the change; the person who prepares a quote cannot approve it; production credentials are never shared.

Official references for this chapter

- Martin Fowler — articles & catalogue — `https://martinfowler.com/`

10. Practical Training Projects and Sign-off

Seven projects, completed **in order**, each in the employee's personal sandbox with a budget alert. Each is reviewed by the Technical Lead against its acceptance criteria. Evidence (repo link, test report, screenshots, handover pack) goes in the portfolio. Projects 1–6 are individual; Project 7 is a team simulation.

#	Project	Level gate	Weeks	Approver
P1	Employee management system	After L3	2	Technical Lead
P2	Inventory management system	After L4	2	Technical Lead + DB Engineer
P3	CRM prototype	After L2/L3	1.5	BA lead + Technical Lead
P4	Billing application	After L4	2	Technical Lead
P5	Role-based enterprise portal	After L5	2	Technical Lead
P6	Multi-module ERP prototype	After L5	3	Technical Lead + PM
P7	Complete simulated client project	After L6	6	Department Head

P1 — Employee Management System

Goal	Master data, departments, documents and basic HR workflows; the first full-stack CRUD with RBAC.
Key requirements	• Employee master (personal, job, contact), departments, designations, reporting manager. • Roles: HR Admin (full), Manager (own team, read), Employee (own profile, read). • Document upload (ID proof) with secure file handling; audit log of edits. • Search/filter/pagination; export to CSV (HR Admin only).
Database design	`department`, `designation`, `employee` (FK dept, desig, manager_id self-FK), `employee_document`, `app_user`, `role`, `user_role`, `audit_log`. Unique `employee_code`; `CHECK (date_of_joining <= current_date)`.
Code deliverables	ASP.NET Core 8 API (layers per Ch 6), EF Core migrations, React + TypeScript screens (list/detail/edit), JWT/cookie auth, OpenAPI.
Testing	≥ 60 % unit coverage on domain/application; integration tests with a PostgreSQL container; role × endpoint permission tests; upload abuse tests.
Handover pack	README (run in < 30 min), ERD, API docs, 1-page admin guide, seed script.

Acceptance criteria	All tests green in CI; manager cannot read another team; audit row exists for each update; uploaded `.exe` renamed `.pdf` is rejected.

```sql
create table employee (
  id              uuid primary key default gen_random_uuid(),
  employee_code   varchar(20)  not null unique,
  full_name       varchar(150) not null,
  email           varchar(254) not null unique,
  department_id   int          not null references department(id),
  designation_id  int          not null references designation(id),
  manager_id      uuid         references employee(id),
  date_of_joining date         not null check (date_of_joining <= current_date),
  status          varchar(12)  not null default 'Active' check (status in
('Active','Inactive','Exited')),
  row_version     bigint       not null default 0,
  created_at      timestamptz  not null default now()
);
create index ix_employee_department on employee(department_id);
create index ix_employee_manager on employee(manager_id);
```

P2 — Inventory Management System

Goal	Stock accuracy through movements: the ledger is truth, balances are derived.
Key requirements	• Items (SKU, UoM, reorder level), warehouses, suppliers. • Goods receipt, issue, transfer, adjustment — each creates immutable movement rows. • Stock on hand per warehouse, low-stock report, stock valuation (weighted average). • Negative stock blocked unless permission granted and audited.
Database design	`item`, `warehouse`, `supplier`, `stock_movement(id, item_id, warehouse_id, qty_signed, unit_cost, ref_type, ref_id, created_at)`, `grn`, `grn_line`, `issue`. Balance = Σ qty_signed (view/materialised table updated in the same transaction).
Code deliverables	Domain service `StockService` (pure, deterministic), application commands, API, simple UI. Golden tests below.
Testing	Hand-calculated tests: receive 10 @ ₹100, receive 10 @ ₹120 → average ₹110; issue 5 → 15 units × ₹110 = ₹1,650. Concurrency test: two simultaneous issues cannot overdraw.
Handover pack	ERD, stock-valuation rule note, restore drill record, user guide.
Acceptance criteria	Σ movements = reported balance for all items; transfer is atomic (both rows or none); concurrency test passes; restore drill witnessed.

```csharp
[Fact] // Weighted average — hand calculation: (10*100 + 10*120)/20 = 110; after issue 5 -> 15 units, value 1650
public void Weighted_average_cost_after_two_receipts_and_one_issue()
{
    var s = new StockLedger();
    s.Receive(qty: 10, unitCost: 100m); s.Receive(qty: 10, unitCost: 120m);
    s.AverageCost.Should().Be(110m);
    s.Issue(5);
    s.OnHand.Should().Be(15); s.Value.Should().Be(1650m);
}
```

P3 — CRM Prototype

Goal	Capture and progress leads to won/lost deals with a visible pipeline.
Key requirements	• Lead capture (web form + manual), owner assignment, duplicate detection by phone/e-mail. • Pipeline stages with configurable probabilities; activities (call, meeting, task); follow-up reminders. • Convert lead → account + contact + opportunity; reports: conversion rate, pipeline value, activity by owner.
Database design	`lead`, `account`, `contact`, `opportunity(stage_id, amount numeric(14,2), expected_close)`, `pipeline_stage`, `activity`, `note`, `user_owner`. Unique partial index on normalised phone.
Code deliverables	REST API, Kanban board UI (React), background reminder job (outbox pattern).
Testing	Story-level acceptance tests (Given/When/Then); duplicate-detection test; forecast calculation golden test: weighted pipeline = Σ amount × probability.
Handover pack	BA pack (AS-IS/TO-BE, stories), ERD, demo script, user guide.
Acceptance criteria	Sales user sees only own leads; manager sees team; forecast numbers match hand calculation on seed data.

P4 — Billing Application

Goal	Accurate, auditable invoicing with tax and payments.
Key requirements	• Customers, items/services, tax rates (from reference table / rule pack — never hard-coded). • Quote → invoice → payment → receipt; credit notes; numbering series per financial year without gaps. • Invoice PDF, payment allocation, outstanding ageing report, reminders.

Database design	`customer`, `item`, `tax_rate`, `invoice`, `invoice_line`, `payment`, `payment_allocation`, `credit_note`, `number_series`. Money `numeric(14,2)`; line total stored; invoice immutable after posting.
Code deliverables	Domain `InvoiceCalculator` (rounding policy documented), posting use case in one transaction with audit and number allocation.
Testing	Golden test: 2 × ₹500 = ₹1,000 + 18 % tax ₹180 = ₹1,180. Rounding tests (half-up per line vs per invoice as stated in SOW). Gap-free numbering test under concurrency.
Handover pack	Tax configuration guide, ERD, sample invoices, runbook for number-series repair (never manual SQL).
Acceptance criteria	Totals match hand calculations; posted invoice cannot be edited (only credited); ageing report reconciles to ledger.

```csharp
[Fact] // 2 x 500 = 1000; tax 18% = 180; total = 1180   (rate comes from tax_rate table, not code)
public void Invoice_total_with_single_tax_rate()
{
    var inv = Invoice.Draft(customerId);
    inv.AddLine(itemId, qty: 2, unitPrice: 500m, taxRate: 0.18m);
    inv.SubTotal.Should().Be(1000m); inv.TaxTotal.Should().Be(180m);
inv.GrandTotal.Should().Be(1180m);
}
```

P5 — Role-based Enterprise Portal

Goal	A portal for customers/vendors/employees with fine-grained access, notifications and audit.
Key requirements	• Login with MFA option; roles and permissions managed by admin; data scope (own company/branch). • Self-service: view orders/invoices, raise tickets, upload documents, download statements. • Notifications (e-mail/in-app) via outbox; full audit trail; admin activity report.
Database design	`app_user`, `role`, `permission`, `role_permission`, `user_scope`, `ticket`, `notification_outbox`, `audit_log`. Row-level scope applied by a central query filter.
Code deliverables	Permission policies, scope filter, outbox dispatcher, admin UI. If tenant isolation is *required*: tenant filter + RLS (Ch 7) — otherwise not built.
Testing	Permission-matrix test (role × endpoint); IDOR tests (user A requests user B's record id); session-expiry and lock-out tests; accessibility check (axe + keyboard).
Handover pack	Threat model (STRIDE table), permission matrix, security checklist, user and admin guides.

Acceptance criteria	Every IDOR attempt returns 404/403 and is logged; no endpoint without authorisation attribute; ZAP baseline scan has no high findings.

P6 — Multi-module ERP Prototype

Goal	Integrate Masters, Sales, Purchase, Inventory, Finance-lite and HR-lite as a modular monolith.
Key requirements	• Module boundaries with public interfaces; shared kernel (numbering, approval, attachments, audit). • Order-to-cash and procure-to-pay flows end-to-end; approval workflow with limits. • Posting to a simple ledger (double entry: debits = credits); dashboard and 5 reports.
Database design	Per-module schemas (`sales.*` , `inventory.*` , `finance.*`), `ledger_entry` (debit, credit, account), `approval_request` , `approval_step` . Cross-module references by ID only.
Code deliverables	Architecture tests forbidding cross-module references; domain events (in-process) with outbox; CI pipeline to staging.
Testing	End-to-end scenario tests: sales order → stock issue → invoice → payment → ledger balances; trial balance debits = credits; performance baseline.
Handover pack	C4 diagrams, 3+ ADRs, module catalogue, runbook, release notes.
Acceptance criteria	Trial balance balances after the scenario suite; architecture tests pass; fresh clone runs in < 30 min.

P7 — Complete Simulated Client Project

Goal	A supervised team (BA, 2 developers, QA, PM) runs all 29 lifecycle stages for a fictional client (e.g. a 40-bed clinic or a 5-branch distributor).
Key requirements	• Role-play client provided by the Technical Lead with realistic messy data and a change request injected in week 4. • All templates (Appendices C–L) completed; all gates G1–G9 executed with sign-offs. • Costing and SOW prepared by the team; margin reviewed by Department Head.
Database design	Designed by the team (3NF ERD, migrations, seed + migrated legacy CSV with reconciliation).

Code deliverables	Deployed to staging and a production-like environment with CI/CD, monitoring and backups.
Testing	Full test plan, UAT executed by the role-play client, security checklist, restore drill, performance baseline.
Handover pack	Complete handover pack (Appendix K), training session delivered, lessons-learned report.
Acceptance criteria	Department Head's panel marks each gate evidence; any missing gate evidence = fail until fixed.

SIGN-OFF STATEMENT

"I have reviewed the evidence for this project against its acceptance criteria. The employee has demonstrated the skills listed and may work on supervised client projects at this level." — Technical Lead (name, date, signature). Sign-off is **supervised-practice readiness**, not independence or certification.

Official references for this chapter

- .NET documentation — `https://learn.microsoft.com/dotnet/`
- ASP.NET Core documentation — `https://learn.microsoft.com/aspnet/core/`
- PostgreSQL documentation — `https://www.postgresql.org/docs/`
- React — Learn — `https://react.dev/learn`
- OWASP Top 10 — `https://owasp.org/www-project-top-ten/`

11. Official Learning Sources

Use **official and current** documentation first. Every chapter ends with the references it relies on. Cite the page and access date in your own documents.

Microsoft / .NET

Source	Link	Used for
.NET documentation	https://learn.microsoft.com/dotnet/	C#, .NET 8, EF Core, WPF entry point
ASP.NET Core documentation	https://learn.microsoft.com/aspnet/core/	APIs, authentication/authorisation, middleware, health checks
SQL Server documentation	https://learn.microsoft.com/sql/	T-SQL, indexes, backup/restore, security
WPF documentation	https://learn.microsoft.com/dotnet/desktop/wpf/	Windows desktop applications
Azure documentation	https://learn.microsoft.com/azure/	Azure hosting and services

Databases

Source	Link	Used for
PostgreSQL documentation	https://www.postgresql.org/docs/	SQL, transactions, indexes, row-level security, backup
MySQL reference manual	https://dev.mysql.com/doc/	MySQL / InnoDB behaviour
Redis documentation	https://redis.io/docs/	Caching, sessions, rate limiting
RabbitMQ documentation	https://www.rabbitmq.com/docs	Queues, exchanges, reliability

Front-end & Python

Source	Link	Used for
React — Learn	https://react.dev/learn	Components, state, forms
TypeScript documentation	https://www.typescriptlang.org/docs/	Typed front-end code
Angular documentation	https://angular.dev/	Angular option
FastAPI documentation	https://fastapi.tiangolo.com/	Python APIs, validation, OpenAPI

Architecture & security

Source	Link	Used for
Martin Fowler — articles & catalogue	https://martinfowler.com/	Patterns, modular monolith, microservice trade-offs
Patterns of Enterprise Application Architecture catalogue	https://martinfowler.com/eaaCatalog/	Unit of Work, Repository, Domain Model
OWASP	https://owasp.org/	Top 10, ASVS, Cheat Sheet Series
OWASP Top 10	https://owasp.org/www-project-top-ten/	Web application risks
OWASP API Security Top 10	https://owasp.org/www-project-api-security/	API risks
OWASP ASVS	https://owasp.org/www-project-application-security-verification-standard/	Security verification checklist
C4 model	https://c4model.com/	Architecture diagrams
OpenAPI Specification	https://spec.openapis.org/oas/latest.html	API contracts
WCAG 2.2	https://www.w3.org/TR/WCAG22/	Accessibility target

Engineering platform

Source	Link	Used for
GitHub Docs	https://docs.github.com/	Repositories, PRs, branch protection, Actions, security features
GitHub Actions	https://docs.github.com/actions	CI/CD workflows
Docker Docs	https://docs.docker.com/	Images, Compose, security

Cloud

Source	Link	Used for
AWS documentation	https://docs.aws.amazon.com/	AWS services
Google Cloud documentation	https://cloud.google.com/docs	GCP services

Cloud pricing (verify at quoting time)

Source	Link	Used for
AWS Pricing Calculator	https://calculator.aws/	Quote hosting
Azure Pricing Calculator	https://azure.microsoft.com/pricing/calculator/	Quote hosting
Google Cloud Pricing Calculator	https://cloud.google.com/products/calculator	Quote hosting
GitHub pricing	https://github.com/pricing	CI minutes and seats

LINK AND PRICE VERIFICATION STATUS

The links above are the canonical official addresses. In the build environment used to generate this edition, outbound access to these sites was restricted, so **links were not live-checked and no vendor price was copied from a vendor page**. The Department Head's office must run a link check and re-date every price-dependent figure before the next release (see Appendix B, week 12 task).

Appendix A — Employee Onboarding Checklist

Field	Value
Employee name	
Role / track	
Start date	
Buddy (peer)	
Technical Lead (mentor)	
Department Head	

Before day 1 (HR + Technical Lead)

- ☐ Offer letter, NDA, IP-assignment and acceptable-use policy signed.
- ☐ Laptop imaged to the standard (disk encryption on, OS updates on, screen lock ≤ 5 min).
- ☐ Company e-mail, chat, project tool and GitHub organisation invitations sent; **MFA mandatory** on every account.
- ☐ Password manager licence assigned; employee told never to store credentials in notes or chat.
- ☐ Sandbox cloud account/budget alert created (≤ agreed monthly cap); no access to client production.
- ☐ Buddy and mentor assigned; 90-day plan (Appendix B) shared; first-week calendar booked.

Day 1

- ☐ Welcome session: company mission, brand, services (Ch 1), the Golden Rules.
- ☐ Policies read and acknowledged: information security, confidentiality, data protection, acceptable use, code of conduct.
- ☐ Workstation set-up verified: Git, VS Code/Visual Studio, .NET 8 SDK, Node LTS, Docker, PostgreSQL client — each with a version check.
- ☐ GitHub: SSH key or credential manager configured; commit signing/e-mail set; first commit to the training repo via PR.
- ☐ Read Chapters 1, 2 and 5 (lifecycle); complete the 10-question orientation quiz.

Week 1

- ☐ Level 1 started; shadow one BA meeting and one stand-up.
- ☐ Complete security awareness module (phishing, passwords, data handling, clean desk).
- ☐ Practise the safety rule: look before delete, backup before change, ask before production.
- ☐ Set up the personal portfolio repository (training evidence).

- [ ] Meet Technical Lead: agree learning goals and the track (BA / Backend / Frontend / Data / QA / Support).

Access matrix (least privilege)

System	Trainee	After L3	After 3 supervised projects
Training repos	Write (PR only)	Write	Write
Client repos	None	Read	Write via PR
Client staging	None	Read	Deploy with approval
Client production	None	None	Support access by ticket only
Cloud sandbox	Own account	Own account	Own account
Costing/finance files	None	None	As role requires

Sign-off

Role	Name	Signature	Date
Employee			
HR			
Technical Lead			
Department Head			

Appendix B — 90-Day Training Plan

Full-time, supervised. Each week ends with a Friday review (30 min) with the mentor. Reduce scope — never skip exit criteria — if time is lost.

Weeks	Focus	Activities	Assessment	Output
1–2	L1 Business systems	Ch 1–2; business fundamentals; order-to-cash map; orientation	Quiz ≥ 80 %; whiteboard demo	Document a small business (L1 capstone)
3–5	L2 Business analysis	Discovery role-plays; process maps; stories & acceptance criteria; change request pricing	Requirement-review defence	BRD + FRD for 4-module mini ERP
4–7	L3 Software engineering (starts week 4, overlaps)	Language fundamentals; OOP; APIs; layers; auth; errors; logging; Git basics	Code review ≥ 80 %	**P1 Employee management system** (weeks 6–7)
8–10	L4 Database engineering	Relational design; SQL; normalisation; transactions; indexes; migrations; backup/restore	Timed SQL practical; restore drill	**P2 Inventory system** (weeks 9–10)
10–11	P3 CRM prototype + P4 start	Stories to working software; pipeline reports	Acceptance demo	**P3 CRM prototype**
11–12	L5 Enterprise architecture	Modular monolith; RBAC/audit; workflow; integration; data migration; multi-tenancy rules	Architecture board (C4 + 3 ADRs)	**P4 Billing app**; P5 started
12	Governance	**Run link/price verification of all sources and rate assumptions (Ch 8, 11)**; update handbook change log	Verification log	Verified source list
12–13	L6 Professional engineering	Pull requests; CI/CD; Docker; cloud deployment; monitoring; OWASP; documentation	Pipeline demo + security checklist	P4 in staging; **P5 Portal**

Weeks 12–13 are the final fortnight of the plan; P6 (ERP prototype) and P7 (simulated client project) continue in the **next 6–9 weeks** as supervised work and complete the certification path of Chapter 10.

Milestones and checkpoints

Day	Checkpoint	Pass criterion
Day 5	Orientation complete	Policies signed; workstation verified; first PR merged
Day 14	Level 1 passed	Quiz ≥ 80 %; L1 capstone approved
Day 35	Level 2 passed	Requirement-review defence passed

Day	Checkpoint	Pass criterion
Day 49	P1 delivered	CI green; acceptance criteria met
Day 70	Level 4 + P2 delivered	Restore drill witnessed
Day 85	Level 5 architecture board	C4 L1–L3 + 3 ADRs accepted
Day 90	Readiness review	Technical Lead + Department Head confirm assignment to a supervised client project

Weekly rhythm

- **Mon:** plan week with mentor; **Tue–Thu:** learn → build → review; **Fri:** demo + retrospective + update training record.
- Daily: 15-minute stand-up (done / next / blocked). Write a 5-line learning log every day.
- Never spend more than 45 minutes stuck without asking — then bring your notes (what you tried, error text, expected vs actual).

Appendix C — Business Requirements Template (BRD)

Document control	
Project / client	
Document ID / version	BRD-<PROJECT>-v0.1
Author / reviewer	
Status	Draft / Review / Approved
Date	

1. Executive summary

- Business problem in 3–5 sentences.
- Objectives and **measurable success criteria** (e.g. invoice time from 15 → 3 min; stock accuracy ≥ 98 %).

2. Business context

- Organisation, locations, departments, headcount, transaction volumes.
- Current systems and tools; pain points with evidence.

3. Scope

- **In scope** (modules/processes).
- **Out of scope** (explicit).
- Assumptions · Constraints · Dependencies.

4. Stakeholders

- Sponsor, decision-maker, process owners, users, IT, finance, external parties (link register T04).

5. Process summary

- AS-IS and TO-BE diagrams for each process (attach); gap list; expected benefit.

6. High-level business requirements

- BR-001… each with business owner, priority (Must/Should/Could/Won't), rationale, success measure.

7. Business rules and calculations

- BRL-001… with worked examples from real data.

8. Reporting needs

- Report name, audience, frequency, data source, sample.

9. Risks and issues

- Risk, likelihood, impact, mitigation, owner.

10. Compliance & data considerations

- Personal/sensitive data handled; retention; consent; regulatory areas to consider with the client's own legal advisers.

11. Glossary and references

- Terms used by the client; documents consulted.

Approval

Role	Name	Signature	Date
Client sponsor			
Business Analyst			
Technical Lead			
Department Head			

Appendix D — Functional Requirements Template (FRD / FRS)

Document control	
Project	
Baseline version	FRS-v1.0 (after G2)
Traceability	BR → FR → Design → Test

1. Module overview

One section per module: purpose, actors, preconditions, main flow, alternate/exception flows, postconditions.

2. Requirement table

ID	Module	Requirement (shall ...)	Priority	Source (BR / interview)	Acceptance criteria (Given/When/Then)	Test ID
FR-001	Sales	The system shall allow a Sales user to create a draft invoice for an active customer.	Must	BR-004	Given an active customer, when the user saves lines, then a draft invoice with number blank is stored.	TC-001
FR-002	Sales	The system shall prevent posting if the credit limit would be exceeded unless a Manager approves.	Must	BRL-002	Given outstanding ₹90,000 and limit ₹1,00,000, when posting ₹20,000, then approval is required.	TC-002
FR-003	All	The system shall record an audit entry for each approval.	Must	NFR-AUD-1	Given an approval, then audit row has user, time, old/new status.	TC-003

3. User-story format

```text
As a <role>
I want <capability>
So that <business benefit>

Acceptance criteria
  Given <context>  When <action>  Then <observable result>
Priority: Must/Should/Could   Estimate: <days>   Depends on: <IDs>
```

4. Non-functional requirements (each measurable)

ID	Category	Requirement	Measure / test
NFR-PERF-1	Performance	List screens respond for 95 % of requests	p95 < 500 ms at 50 concurrent users (load test)
NFR-AVAIL-1	Availability	Business-hours availability	≥ 99.5 % monthly (monitoring report)
NFR-SEC-1	Security	No critical/high findings at release	Scan + manual review
NFR-BKP-1	Backup	RPO 24 h (or as agreed), RTO 4 h	Quarterly restore drill
NFR-ACC-1	Accessibility	WCAG 2.2 AA target for staff screens	axe + keyboard test
NFR-AUD-1	Audit	Append-only audit of create/update/delete/approve	Audit tests

5. Input/output specification

Field	Type	Length	Mandatory	Validation	Source/Default
customer_name	text	150	Yes	trim; no control chars	—
invoice_date	date	—	Yes	≤ today + 0; within open period	today

6. Roles and permissions matrix

Action	Clerk	Manager	Accountant	Admin	Auditor
Create invoice	✔	✔	—	—	—
Approve invoice	—	✔	—	—	—
View audit log	—	—	—	✔	✔

Approval

Role	Name	Signature	Date
Client SPOC			
Business Analyst			
Technical Lead			

Appendix E — Statement of Work (SOW) Template

LEGAL REVIEW

This template is a business structure, not legal advice. Legal counsel must review the final contract wording before signature.

1. Parties and project

- Client legal name/address; VEL Software Solutions; project name; SOW number; effective date.

2. Background and objectives

- Reference the signed BRD/FRS baseline (version, date).

3. Scope of services

- Modules and features by FR ID range.
- Environments (dev/staging/production) and number of users/branches.
- Integrations listed by name and direction.

4. Deliverables

- Software, source code (per IP clause), documentation, user manuals, training, handover pack.

5. Out of scope

- Anything not listed in Section 3, including hardware, third-party licences, data entry/cleansing beyond the agreed allowance, regulatory certification.

6. Assumptions and client responsibilities

- Timely access to stakeholders and data; named SPOC; 5-working-day feedback SLA; client-owned licences/accounts; test data availability.

7. Timeline and milestones

- Milestone table with dates tied to gates G2–G9; delay caused by late client input extends the schedule day-for-day.

8. Commercials

- Price (ex-tax) by module; payment schedule (default 30/30/30/10 at G3/G4/G7/G9); recurring hosting/support fees; taxes; invoice terms; late-payment terms.

9. Acceptance

- UAT procedure, criteria (Ch 5 Stage 23), deemed acceptance after N working days of no response or on production use.

10. Change control

- Written change requests priced per Ch 8; no work on changes without approval.

11. Warranty and support

- Defect warranty period; severity definitions P1–P4 and response/resolution targets; support hours; exclusions.

12. Intellectual property and confidentiality

- Ownership of bespoke code, retained rights in reusable components and tools, licence to client, NDA.

13. Data protection and security

- Roles (controller/processor), data handled, security measures summary, breach notification process, data return/deletion at exit. No guarantee of absolute security or certification is given.

14. Limitation of liability, termination, governing law

- As approved by Legal.

Approval

Role	Name	Signature	Date
Client authorised signatory			
VEL authorised signatory (Department Head/Director)			

Appendix F — Database Design Checklist

Modelling

- [ ] Entities and relationships trace to FR IDs.
- [ ] 3NF unless a documented, measured reason to denormalise.
- [ ] Naming convention applied (snake_case, singular tables, `_id` keys).
- [ ] Master vs transaction vs reference tables separated.

Integrity

- [ ] Every table has a primary key.
- [ ] Foreign keys exist for every relationship (with defined ON DELETE behaviour).
- [ ] UNIQUE constraints for natural keys (e.g. invoice number per period).
- [ ] CHECK constraints for ranges/statuses; NOT NULL by default.
- [ ] Money in `numeric/decimal(14,2)`, never float; timestamps in UTC (`timestamptz` / `datetime2`).

Concurrency & transactions

- [ ] Transaction boundary per use case documented.
- [ ] Row-version/optimistic concurrency on editable records.
- [ ] Gap-free number series use a locked counter row, not `MAX()+1`.

Performance

- [ ] Indexes on FKs and common filters; each justified by a query plan.
- [ ] No `SELECT *` in application code; pagination on all list queries.
- [ ] Reporting queries tested with production-scale data.

Security

- [ ] Least-privilege DB accounts (app ≠ migration ≠ admin).
- [ ] Sensitive columns identified; encryption at rest/in transit; masking in non-production.
- [ ] Audit table append-only (no UPDATE/DELETE grants).
- [ ] If multi-tenant is *explicitly required*: tenant_id + isolation tests.

Migrations

- [ ] Forward-only versioned scripts; applied scripts never edited.
- [ ] Rehearsed on production-sized copy; lock duration measured.
- [ ] Rollback = restore or compensating migration documented.

Backup & recovery

- [ ] Backup schedule and retention documented; off-site encrypted copy.
- [ ] RPO/RTO agreed with client.
- [ ] **Restore tested** and recorded (date, duration, result).

Documentation

- [ ] ERD current; data dictionary lists each column's meaning, type and source.

Approval

Role	Name	Signature	Date
Database Engineer			
Technical Lead			

Appendix G — Code Review and Security Checklist

Pull-request review

- [ ] Linked to a story/defect; scope is small and focused.
- [ ] Tests added/updated and meaningful; CI green.
- [ ] No business logic in controllers/UI; layering respected.
- [ ] Validation at edge, domain and DB where relevant.
- [ ] Errors handled; no swallowed exceptions; safe messages to clients.
- [ ] Logging has correlation ID and **no secrets/PII**.
- [ ] Docs/ADR/OpenAPI updated.
- [ ] Migrations reviewed by Database Engineer.

Security (OWASP Top 10 aligned)

- [ ] Access control: server-side checks at endpoint and object level (A01).
- [ ] Cryptography: TLS everywhere; passwords hashed with a modern adaptive algorithm (Argon2id/bcrypt/PBKDF2); keys in a secret store (A02).
- [ ] Injection: parameterised queries; encoded output (A03).
- [ ] Insecure design: threat model and abuse cases exist (A04).
- [ ] Misconfiguration: debug off, headers set, default accounts removed, CORS allow-list (A05).
- [ ] Vulnerable components: dependency/SBOM scan and update plan (A06).
- [ ] AuthN failures: MFA option, lock-out, session expiry, no credential stuffing paths (A07).
- [ ] Integrity: signed builds/pinned actions; no unsafe deserialisation (A08).
- [ ] Logging & monitoring: security events logged and alerted (A09).
- [ ] SSRF: outbound allow-list for URL fetchers (A10).
- [ ] Secret scan passes; no secrets in repo history.
- [ ] File upload rules (Ch 7) applied.

Approval

Role	Name	Signature	Date
Reviewer			
Technical Lead			

Appendix H — QA Checklist

Test planning

- [ ] Test plan approved: scope, levels, environments, data, tools, entry/exit criteria, risks.
- [ ] Every FR and NFR traced to ≥ 1 test case (traceability matrix).
- [ ] Defect severity scale agreed (Critical/High/Medium/Low) with fix targets.

Execution

- [ ] Unit tests pass; coverage target met on domain/application code (default ≥ 70 %).
- [ ] Integration tests pass against a real database container.
- [ ] System tests cover each end-to-end business scenario incl. exception paths (cancel, reverse, reject).
- [ ] Regression suite executed on the release candidate.
- [ ] Boundary, negative and invalid-input tests for every form and API.
- [ ] Permission tests: every role × sensitive action; unauthenticated access rejected.
- [ ] If multi-tenant: cross-tenant read/write/report/job tests pass.
- [ ] Concurrency tests: double-submit, simultaneous edits, number-series.
- [ ] Calculations verified against hand-calculated golden values from the client's real examples.
- [ ] Data-integrity: totals, counts and ledger balances reconcile after test scenarios.
- [ ] Performance test meets NFRs (p95, concurrency, report duration).
- [ ] Security scan (SAST/dependencies/secrets/container) clean of critical/high.
- [ ] Accessibility: keyboard-only walk-through; axe/Lighthouse results recorded.
- [ ] Cross-browser/device matrix tested as per NFR.
- [ ] Backup restore tested; log/alert samples verified.
- [ ] Print/PDF/export outputs match approved samples.

Exit criteria for Release Candidate (G6)

- [ ] Zero open Critical/High defects; Medium defects have agreed workaround or date.
- [ ] All test evidence stored in `tests/` or QA tool; report signed.
- [ ] Release notes and known-issues list prepared.

Test run	Date	Build/tag	Total	Passed	Failed	Blocked	Tester

Approval

Role	Name	Signature	Date
QA Engineer			
Technical Lead			

Appendix I — User Acceptance Testing (UAT) Checklist

Preparation

- [ ] Release candidate deployed to **staging** with production-like configuration and realistic (masked) data.
- [ ] UAT scripts derived from acceptance criteria; each has steps, test data and expected result.
- [ ] UAT testers named by the client (one per role) and trained for 1–2 hours; UAT window agreed.
- [ ] Defect logging channel and triage rhythm (daily, within 24 h) agreed.
- [ ] Entry criterion: G6 passed.

Execution

- [ ] Each script executed by a real user, not by the project team.
- [ ] Results recorded: Pass / Fail / Blocked with evidence (screenshot, record ID).
- [ ] Defects triaged: *defect* (fix now) vs *change request* (price) vs *training issue*.
- [ ] Fixes retested by the original tester; regression by QA.
- [ ] Business reports and printed documents checked against approved samples.
- [ ] Role permissions verified by the process owners.

UAT script format

ID	Story/FR	Role	Steps	Test data	Expected result	Actual	Status	Tester/date
UAT-001	FR-002	Manager	1. Open invoice INV-D-1042 2. Click Post	Customer C-17, limit ₹1,00,000, outstanding ₹90,000	System asks for approval; audit row created			
UAT-002								

Exit and sign-off (G7)

- [ ] 100 % of Must scripts passed; no open Critical/High defects.
- [ ] Accepted-defect list (Medium/Low) signed with fix dates.
- [ ] Training completion ≥ 90 % of users scheduled.
- [ ] Go-live readiness meeting held.

ACCEPTANCE STATEMENT

"The undersigned confirm that the delivered system has been tested against the agreed acceptance criteria in SOW <no.> and is accepted for production use, subject to the listed defects."

Approval

Role	Name	Signature	Date
Client sponsor			
Client SPOC			
Project Manager			
Business Analyst			

Appendix J — Deployment Checklist (Staging and Production)

Pre-deployment (T-3 to T-1 days)

- [ ] Change record approved; window agreed with client; stakeholders informed.
- [ ] Release candidate tagged; artifact/image digest recorded; SBOM and scans attached.
- [ ] Environment configuration reviewed (no secrets in repo; values in secret store); configuration diff vs staging checked.
- [ ] Database migrations rehearsed on a copy of production; duration measured.
- [ ] **Verified backup** taken and restore tested; backup location and time recorded.
- [ ] Rollback plan written with decision point and time limit; data-rollback approach defined.
- [ ] Monitoring dashboards and alerts ready; on-call contacts confirmed.
- [ ] Runbook reviewed by a second engineer.

Deployment

- [ ] Freeze: no other changes to the environment.
- [ ] Maintenance notice shown if downtime is expected.
- [ ] Deploy via pipeline (no manual file copy).
- [ ] Run migrations; confirm version number.
- [ ] Smoke tests: login, one transaction per module, reports, integrations, scheduled jobs.
- [ ] Check logs/metrics for errors; compare latency/error rate to baseline.

Post-deployment

- [ ] Client SPOC confirms smoke test on production.
- [ ] Hyper-care schedule active (24–72 h); issue channel open.
- [ ] Release notes sent; documentation updated.
- [ ] Change record closed with result and timings.
- [ ] Backups scheduled and first scheduled backup verified.
- [ ] Retrospective logged if any deviation.

Rollback triggers (example)

- Smoke test failure in a Must function; error rate > 2× baseline for 10 minutes; data-integrity discrepancy; security issue discovered.

Approval

Role	Name	Signature	Date
Deployer			
Approver (Technical Lead)			
Client sponsor (production)			

Appendix K — Client Handover Checklist

Deliverables

- ☐ Source code repository ownership/licence per SOW; branch protection and access transferred.
- ☐ Architecture document, ADRs, ERD and data dictionary.
- ☐ API documentation (OpenAPI) and integration contracts.
- ☐ Deployment guide, runbooks (start/stop, backup/restore, rollback, incident), monitoring guide.
- ☐ User manuals per role; admin manual; training recordings.
- ☐ Security checklist, scan report, accepted-risk register.
- ☐ Test evidence summary and UAT sign-off.
- ☐ Configuration list (environment variables by name — **no secrets in the document**).

Access and ownership

- ☐ Cloud accounts, domains, certificates and DNS in the client's name; renewal dates recorded.
- ☐ Credentials transferred via approved secret vault; our temporary access removed or converted to ticket-based support access.
- ☐ Admin users created for client; default/test accounts disabled.
- ☐ Third-party licences/API keys registered to the client.

Knowledge transfer

- ☐ Admin/IT training session held and recorded.
- ☐ Super-users identified and trained.
- ☐ Support process explained: how to raise tickets, severity levels, contacts, hours.
- ☐ Known issues and roadmap shared.

Commercial closure

- ☐ Final invoice raised per milestones.
- ☐ Warranty start and end dates recorded.
- ☐ Support/maintenance contract signed or declined in writing.
- ☐ Customer satisfaction survey sent; lessons-learned session held internally.

HANDOVER CERTIFICATE

“The client acknowledges receipt of the deliverables listed above and accepts responsibility for operating the system, subject to the warranty and support terms in SOW <no.>.”

Approval

Role	Name	Signature	Date
Client sponsor			
Client IT owner			
Project Manager			
Technical Lead			
Department Head			

Appendix L — Project Costing Model Specification

Specification for the department's estimating workbook (spreadsheet or small web tool). Implementation must reproduce the worked examples in Chapter 8 to the rupee — those are the acceptance tests.

Sheets

Sheet	Purpose	Key columns / cells
`Inputs`	Global assumptions (versioned, dated).	Cost/day by role · target margin · PM ratio · phase ratios · contingency by size · overhead/month · management fee % · support/upgrade % · tax note
`RateCard`	Billing rates derived from cost.	Role · cost/day · rate/day = cost ÷ (1 − margin)
`Modules`	Three-point estimates.	Module · FR range · O · M · P · expected = (O+4M+P)/6 · risk flag
`Phases`	Effort by lifecycle component.	Component · basis · days · cost/day · cost
`Integrations`	Per interface.	Name · complexity (3/6/12 d) · vendor · sandbox availability
`Migration`	Per data source.	Source · tables · quality · days
`Hosting&Licences`	Quoted from **official calculators/vendor quotes**.	Service · SKU · region · unit price · qty · months · source URL · date verified · screenshot ref
`Summary`	Price, margin, schedule.	Labour · contingency · licences · hosting · direct · price · margin % · payment schedule
`Recurring`	Annual support, hosting, upgrades.	Support % · hosting × 12 × (1+fee) · upgrade block
`BreakEven`	Department economics.	Overhead/month · contribution per project · projects/year
`ChangeRequests`	CR log and pricing.	CR ID · roles/days · assessment fee · post-freeze premium · status

Formulas (authoritative)

```TEXT
LabourCost      = Σ ( Days[c] × CostPerDay[c] )                  over all components c incl.
Project Management
PM_Days         = PM_Ratio × Σ Days[c ≠ PM]
Contingency     = LabourCost × Risk%                             (10 % small, 15 % medium, 20 %
enterprise)
DirectCost      = LabourCost + Contingency + Licences + BuildHosting
Price           = DirectCost ÷ (1 − TargetMargin)                 (TargetMargin default 0.40; floor
0.30)
GrossMargin%    = (Price − DirectCost) ÷ Price
ModulePrice[m]  = (Price − SharedPrice − LicHostPrice) × DevDays[m] ÷ Σ DevDays
SharedPrice     = (IntegrationCost + MigrationCost) × (1 + Risk%) ÷ (1 − TargetMargin)
AnnualSupport   = Support% × Price ;  AnnualHosting = Monthly × 12 × (1 + MgmtFee)
BreakEven       = 12 × OverheadPerMonth ÷ (Price − DirectCost)       projects per year
ChangeRequest   = 0.5×BA_rate + Σ(days × rate) × (1 + 0.20 if after freeze)
```

Controls

- Inputs sheet is **versioned and dated**; changing a rate requires Department Head approval.
- Every price in `Hosting&Licences` carries source URL, SKU, region, date verified — no row without evidence.
- Technical Lead reviews module days; Finance reviews margin; both initial the Summary sheet.
- Model must fail validation if: margin < floor, contingency missing, any of the 17 cost components has zero days without a written reason.
- Export a PDF quote snapshot stored with the SOW; never overwrite a quoted version.

Acceptance tests for the model

Case	Expected (ex-tax)
Small example (Ch 8)	Price ₹6,82,567 · margin ₹2,73,027 (40 %)
Medium example	Price ₹27,08,833 · margin ₹10,83,533
Enterprise example	Price ₹93,87,033 · margin ₹37,54,813
Change request (Medium, 1 BA + 4 Dev + 2 QA + 0.5 DB)	₹64,333

Approval

Role	Name	Signature	Date
Technical Lead			
Finance			
Department Head			

Appendix M — Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — properties of reliable transactions.
ADR	Architecture Decision Record — short document of a decision, options and consequences.
API	Application Programming Interface — contract for software to talk to software.
AS-IS / TO-BE	Current process / designed future process.
Audit trail	Append-only record of who did what and when.
BRD / FRS	Business / Functional Requirements Specification.
CI/CD	Continuous integration / delivery: automated build, test and deploy.
CR	Change request after requirements freeze.
Idempotency	Repeating an operation has the same effect as doing it once.
Modular monolith	One deployable application with strict internal module boundaries.
Multi-tenancy	One system serving multiple isolated customers (tenants).
NFR	Non-functional requirement: performance, security, availability etc.
Outbox pattern	Write event to a table in the same transaction, publish afterwards for reliability.
RACI	Responsible, Accountable, Consulted, Informed.
RBAC	Role-based access control.
RPO / RTO	Maximum acceptable data loss / maximum acceptable downtime.
SOW	Statement of Work — contractual scope, price and terms.
SPOC	Single point of contact.
UAT	User acceptance testing by the client.
WCAG	Web Content Accessibility Guidelines.